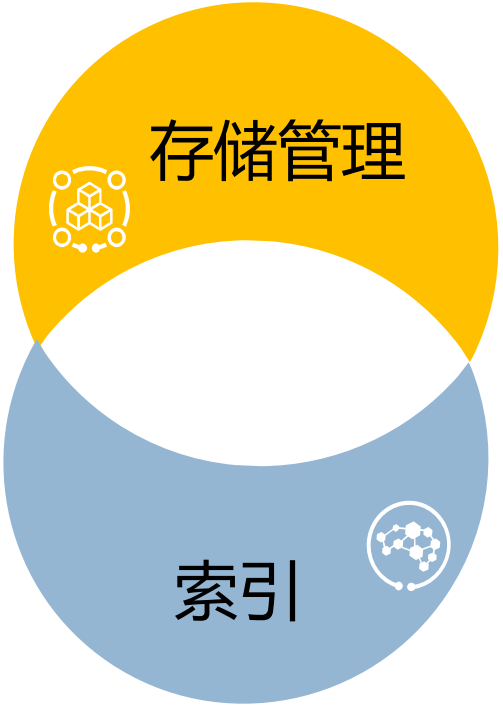
数据库系统概论

Introduction to Database Systems

第9章　关系数据库存储管理



存

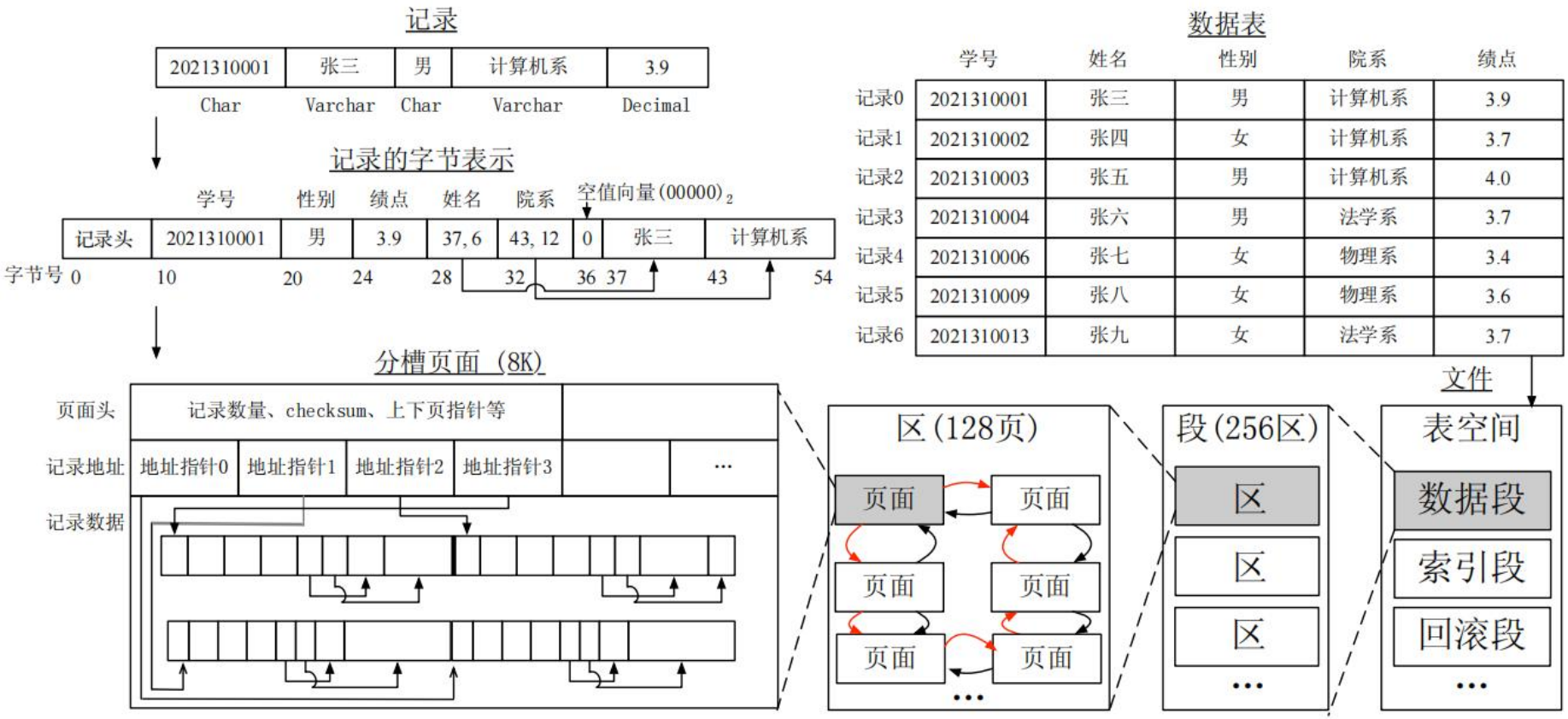
① 物理存储系统
② 数据组织
③ 元数据存储
④ 缓冲区

取

① 顺序文件上的索引
② 辅助索引
③ B+树索引
④ 哈希索引
⑤ Bitmap索引

- ❖ 9.1 数据组织
- ❖ 9.2 索引结构
- ❖ 本章小结

数据组织


记录
2021310001
张三
男
计算机系
3.9
Char
Varchar
Char
Varchar
Decimal
记录的字节表示
学号
性别
绩点
姓名
院系
空值向量(00000)2
记录头
2021310001
男
3.9
37, 6
43, 12
0
张三
计算机系
字节号 0
10
20
24
28
32
36 37
43
54
分槽页面 (8K)
页面头
记录数量、checksum、上下页指针等
记录地址
地址指针0
地址指针1
地址指针2
地址指针3
...
记录数据
区(128页)
页面
页面
页面
页面
页面
页面
...
段(256区)
区
区
区
...
表空间
数据段
索引段
回滚段
...
数据表
学号
姓名
性别
院系
绩点
记录0
2021310001
张三
男
计算机系
3.9
记录1
2021310002
张四
女
计算机系
3.7
记录2
2021310003
张五
男
计算机系
4.0
记录3
2021310004
张六
男
法学系
3.7
记录4
2021310006
张七
女
物理系
3.4
记录5
2021310009
张八
女
物理系
3.6
记录6
2021310013
张九
女
法学系
3.7
文件

- 以最优的形式在外存上组织、存放庞大数据集
  - 最优：
    - 存储效率高，节省存储空间
    - 存取效率高，访问速度快，代价小

- 从软件角度缓解I/O瓶颈通常采用的措施
  - 使用缓冲区，减少物理I/O的发生——需要考虑数据一致性问题
  - 利用合适的数据组织方式，减少I/O的发生
  - 有针对性的预读或使用系统提供的异步I/O能力，把等待I/O的时间并行做其他任务

- ❖ **9.1.1** 数据库的逻辑组织方式与物理组织方式
- ❖ **9.1.2** 记录表示
- ❖ **9.1.3** 块的组织
- ❖ **9.1.4** 关系表的组织

- ❖9.1.1 数据库的逻辑组织方式与物理组织方式
- ❖9.1.2 记录表示
- ❖9.1.3 块的组织
- ❖9.1.4 关系表的组织

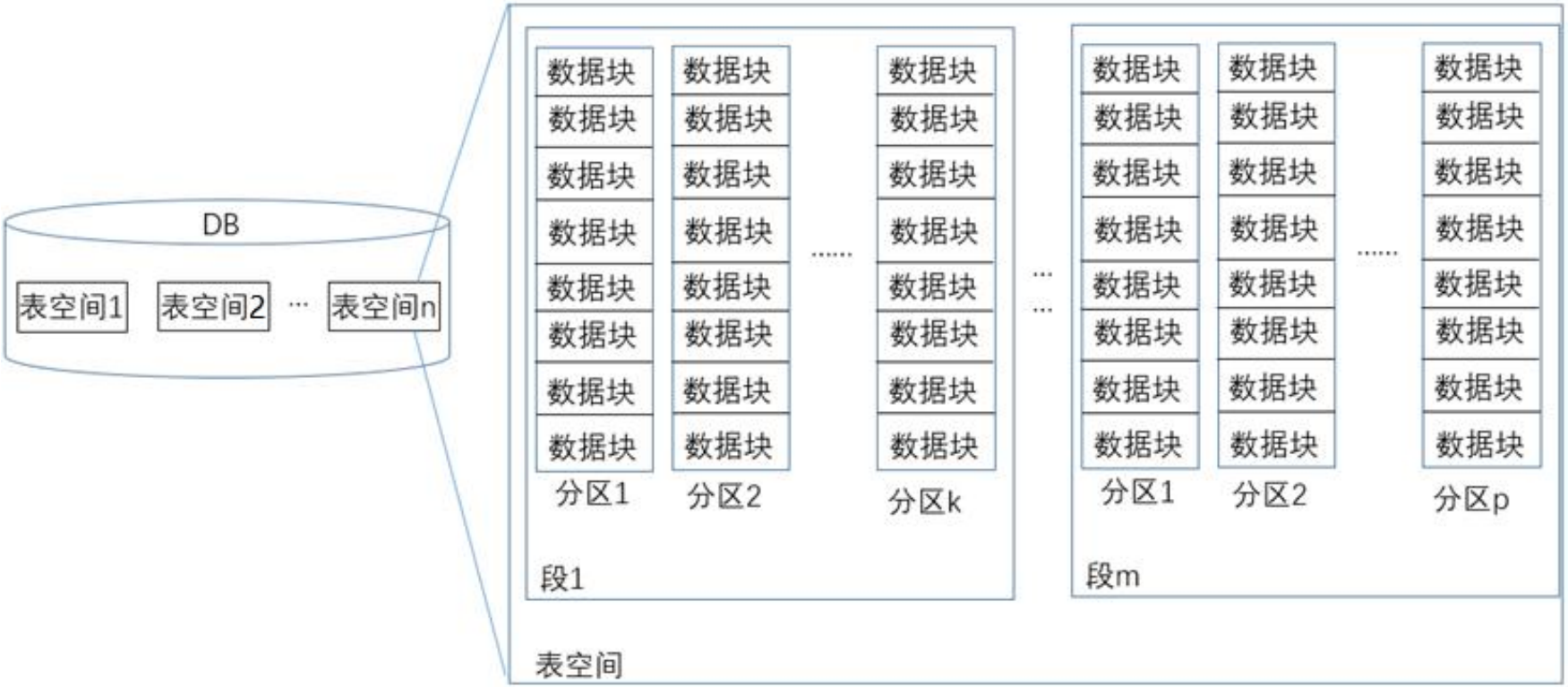
❖数据存储的管理方式

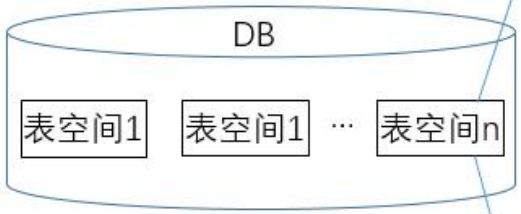
- 方式一：每个DB对象对应一个操作系统文件
  - e.g. PostgreSQL, Kingbase ES
- 方式二：整个DB对应一个或若干个文件（段页式）
  - e.g. Oracle, SQL Server

❖数据存储的管理方式

- 方式一：每个DB对象对应一个操作系统文件
  - e.g. PostgreSQL, Kingbase ES
- 方式二：整个DB对应一个或若干个文件（段页式）
  - e.g. Oracle, SQL Server

- 数据库的逻辑组织方式：
  - 表空间----段----分区----数据块


DB
表空间1
表空间2
…
表空间n
数据块
分区1
分区2
分区k
段1
分区1
分区2
分区p
段m
表空间



❖ 表空间(逻辑设备)

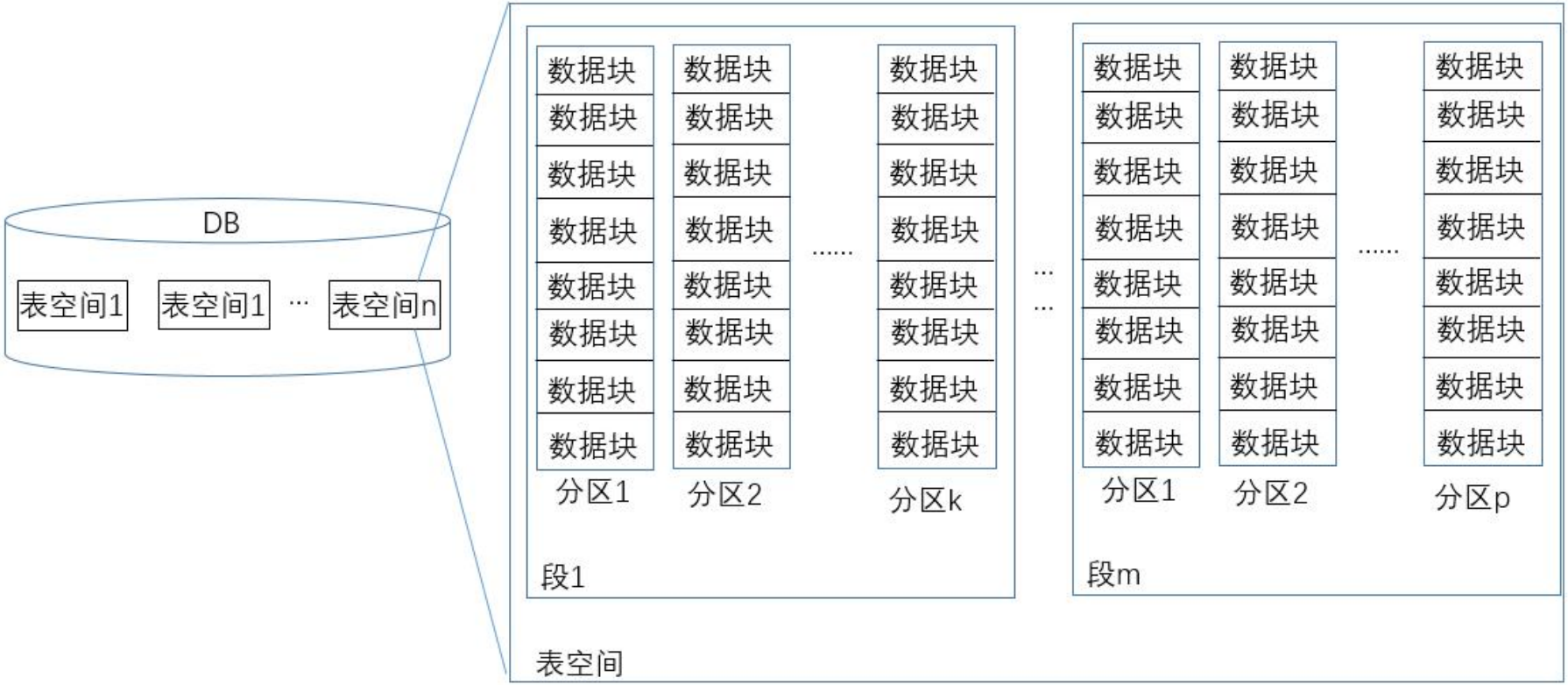
- 对应磁盘上一个或多个物理数据文件
- 可以有多个表空间，逻辑地和物理地组织数据库中的数据存储。
- 系统表空间、联机表空间、脱机表空间、永久表空间、临时表空间

❖ 数据文件

表空间

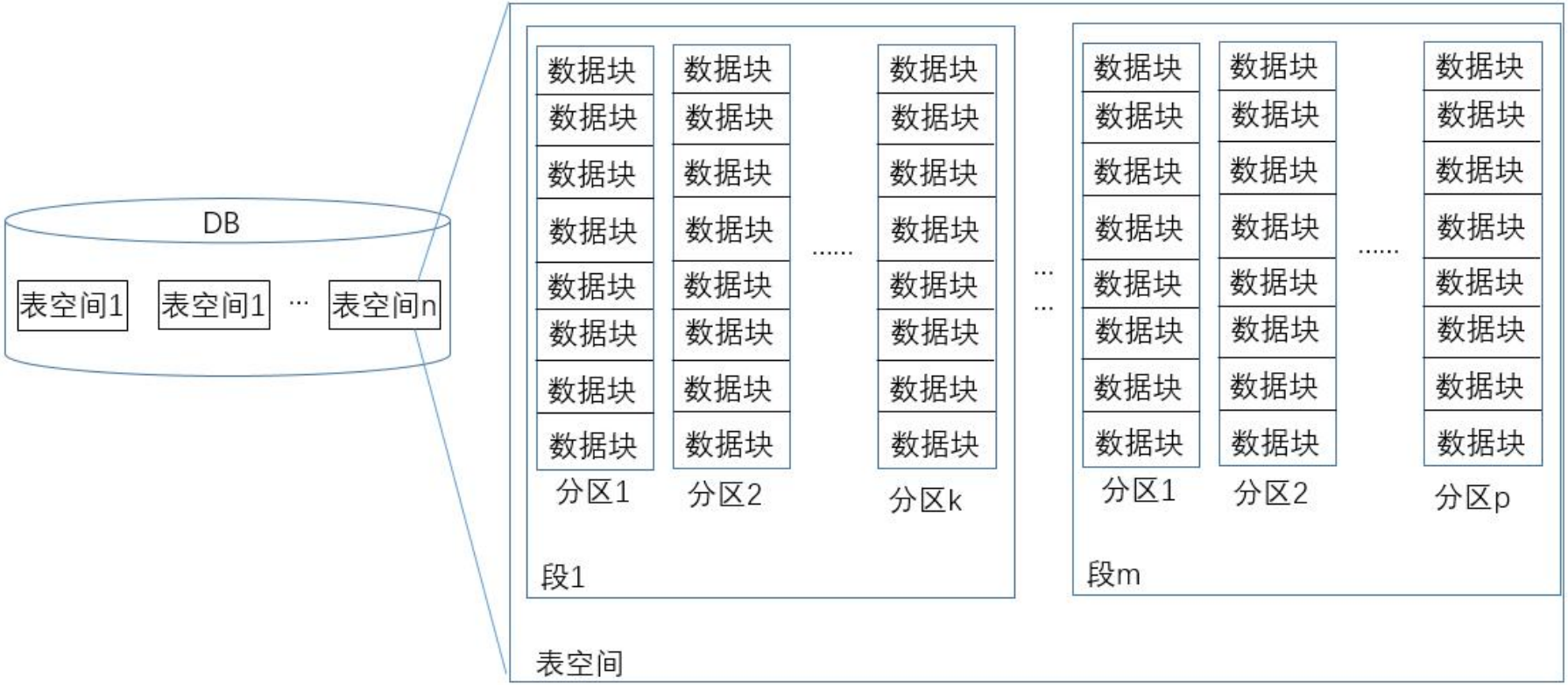
❖段

- 由多个分区组成。
- 数据段、索引段、LONG段、回滚段、临时段等



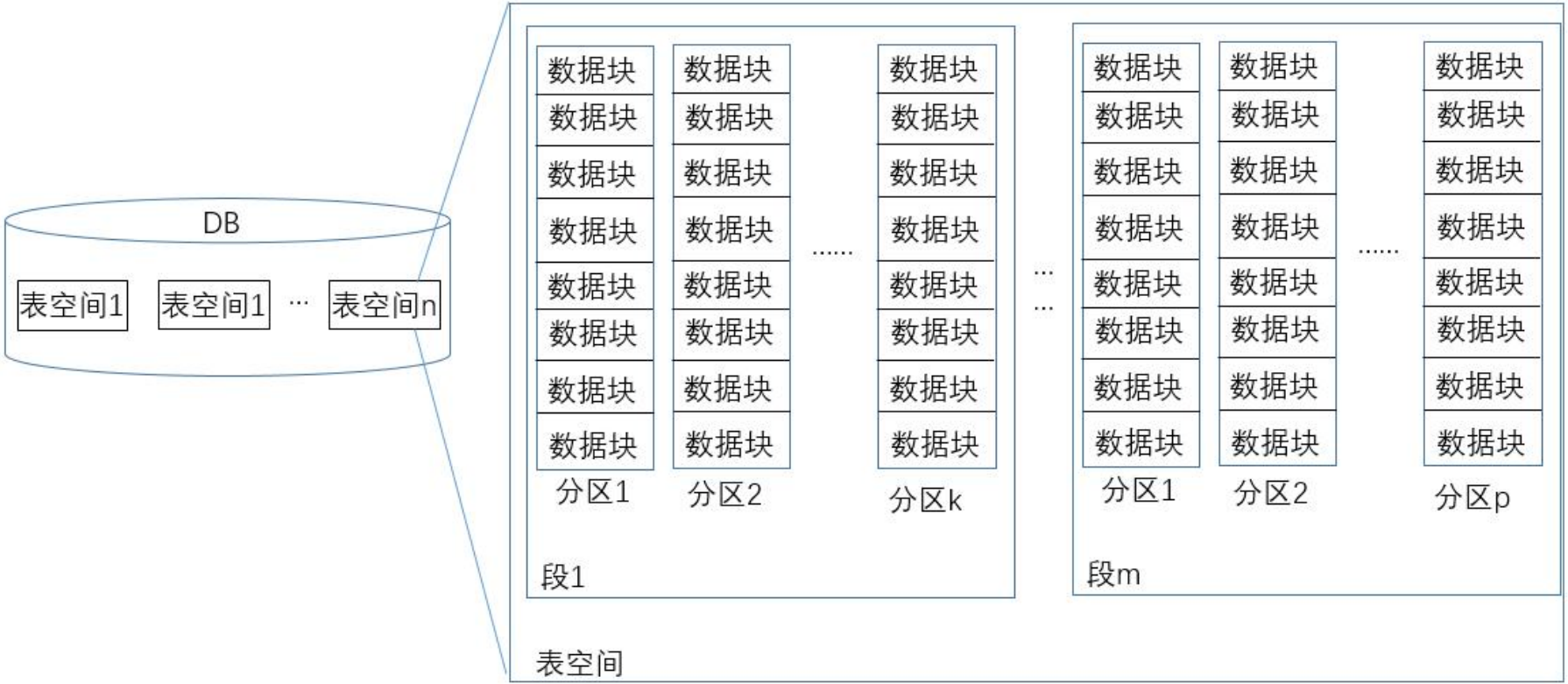
❖分区

■ 由一组连续的数据块组成



❖数据块

- 数据块是磁盘存取单元
- 数据块的大小必须等于服务器操作系统块的大小或倍数

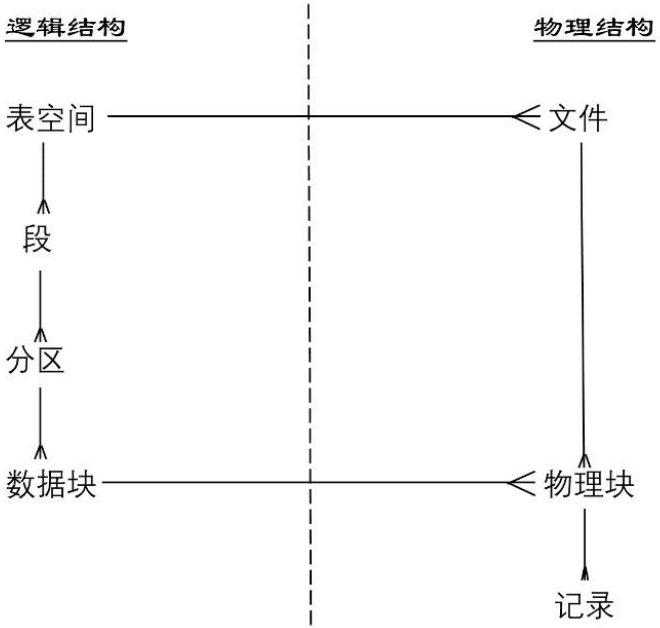


❖ 数据库的物理组织方式：

- 文件----块----记录

- 文件
  - 操作系统文件
- 块
  - 每个文件物理上分成定长的存储单元，即操作系统的物理块。
  - 存储分配和I/O的基本单位
- 记录
  - 物理块中存放的多条元组（记录）

❖数据库逻辑组织方式与物理组织方式的对应关系



- ❖9.1.1 数据库的逻辑组织方式与物理组织方式
- ❖9.1.2 记录表示
- ❖9.1.3 块的组织
- ❖9.1.4 关系表的组织

- 元组存储
  - 形式一：定长记录存储
  - 形式二：变长记录存储

- 元组存储
  - 形式一：定长记录存储
  - 形式二：变长记录存储

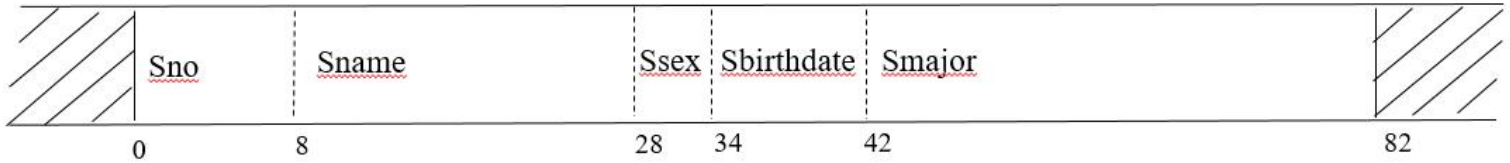
- 定长记录存储
  - 关系表中的每条元组占据相同大小的空间
  - 变长字段以定长形式存储，预留最大长度空间

- 定长记录存储
  - 例：

```
CREATE  TABLE  Student (
   Sno  CHAR(8) PRIMARY KEY,
   Sname CHAR(20) UNIQUE,
   Ssex  CHAR(6),
   Sbirthdate  Date,
   Smajor  VARCHAR(40)
 );
```

❖定长记录存储

■例：



VARHCHAR类型属性
分配最大长度空间

❖有些系统要求数据在内存中的存取地址

- 在外存中保证各字段的起始地址是4或8的倍数
- 例：

1. 每个字段都是从8的倍数的地址起始
2. Sname是20字节，但分配了24字节
3. Ssex定义了6字节，但分配了8字节

- 定长存储的优劣
  - 优势
    - 快速定位到记录及其属性的物理位置
    - 增删改比较方便快捷
  - 劣势
    - 浪费存储空间

- 元组存储
  - 形式一：定长记录存储
  - 形式二：变长记录存储

- 变长记录
  - 存储要求
    - 能快速访问一条记录
    - 能快速访问记录中所有属性（定长个变长属性）
  - 变长记录存放的三种方式

❖方式一：变长字段前加长度

- 在每条记录的头部记录该条记录的长度
- 在记录的每个变长字段前记录该字段的长度
- 例：

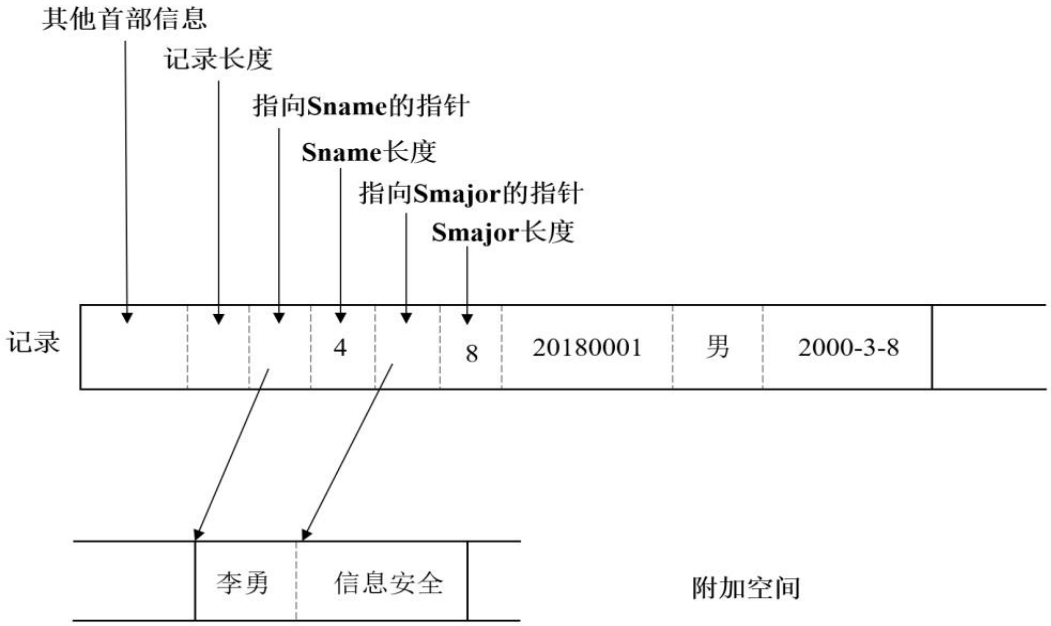
❖方式二

- 先存放定长字段，再存放变长字段
- 第一个变长字段紧随定长字段，从第二个变长字段开始，在记录首部用指针（偏移量）指向变长字段
- 例：

- 方式三
  - 将定长字段与变长字段分开存储在不同的块中
  - 多用于**BLOB**等类型数据存储
  - 若经常访问定长字段，可减少数据存取时的**I/O**数量

❖方式三

■例：


其他首部信息
记录长度
指向Sname的指针
Sname长度
指向Smajor的指针
Smajor长度
记录
4
8
20180001
男
2000-3-8
李勇
信息安全
附加空间

- 可变格式记录的用途
  - 不同信息源集成。通过只列出非空字段来节省空间。
  - 具有非常灵活的模式的记录。如一条记录的许多字段会重复或根本不出现

- ❖9.1.1 数据库的逻辑组织方式与物理组织方式
- ❖9.1.2 记录表示
- ❖9.1.3 块的组织
- ❖9.1.4 关系表的组织

- 定长记录存储的块组织
- 变长记录存储的块组织

- 定长记录存储的块组织
  - 表中元组依次存放在块中
  - 首部+记录+空闲空间
    - 首部：块ID、最后一次修改和访问该块的时间戳、每条元组在块内的偏移量、空闲空间头指针

❖定长记录存储的块组织

■例：

块头					
记录0	20180001	李勇	男	2000-3-8	信息安全
记录1	20180002	刘晨	女	1999-9-1	计算机科学与技术
记录2	20180003	王敏	女	2001-8-1	计算机科学与技术
记录3	20180004	张立	男	2000-1-8	计算机科学与技术
记录4	20180005	陈新奇	男	2001-11-1	信息管理与信息系统
记录5	20180006	赵明	男	2000-6-12	数据科学与大数据技术
记录6	20180007	王佳佳	女	2001-12-7	数据科学与大数据技术
空闲空间					

物理块

- 定长记录存储的块维护
  - 增：在空闲空间直接插入新元组
  - 改：直接在原位置修改
  - 删：回收空间，将空闲空间加入空闲空间链表
    - 例：在**Student**表中删除**2018003**和**2018006**两条元组

●删除后：

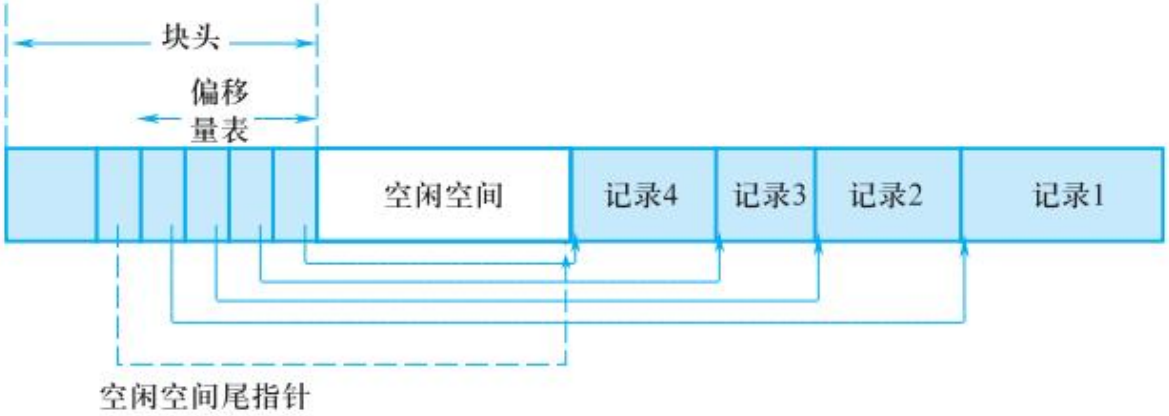
块头
记录0
20180001
李勇
男
2000-3-8
信息安全
记录1
20180002
刘晨
女
1999-9-1
计算机科学与技术
记录2
记录3
20180004
张立
男
2000-1-8
计算机科学与技术
记录4
20180005
陈新奇
男
2001-11-1
信息管理与信息系统
记录5
记录6
20180007
王佳佳
女
2001-12-7
数据科学与大数据技术
空闲空间
物理块

- ❖定长记录存储的块组织
- ❖变长记录存储的块组织

- 变长记录存储的块组织
  - 表中元组从块的尾部连续存放
  - 首部+空闲空间+记录
    - 首部：各记录的指针（块内偏移量）、空闲空间为尾指针（块内偏移量）

❖变长记录存储的块组织

■ 例：



- 变长记录存储块的维护
  - 增
    - 从空闲空间尾部分配空间
    - 在偏移量表中记录改元组的起始位置
    - 调整空闲空间尾指针

- 变长记录存储块的维护
  - 删
    - 在偏移量表中为该元组指针置删除标记
    - 释放元组空间，移动物理位置在其前面的元组（保证空间连续）
    - 修改指针
      - 被移动元组在偏移量表中的指针
      - 空闲空间尾指针

- 变长记录存储块的维护
  - 改
    - 在原位置修改
    - 若修改后记录在原位置放不下，会带来记录的迁移

- ❖9.1.1 数据库的逻辑与物理组织
- ❖9.1.2 记录表示
- ❖9.1.3 块的组织
- ❖9.1.4 关系表的组织

- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - B+树存储
  - 哈希存储

- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - B+树存储
  - 哈希存储

- 堆存储
  - 表中的一条记录可以存放在该表的任何块中，没有顺序要求
  - 插入元组时，在该表的块中找到合适的空闲空间即可
  - 如果没有足够的空闲空间，就为该表申请新的块

- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - B+树存储
  - 哈希存储

- 顺序存储
  - 一个表中的各条记录根据指定的属性或属性组的取值大小顺序的存放
  - 同一个表的不同块中通过指针链接实现有序

❖顺序存储的优劣

- 优势：可以高效地处理按排序属性（组）进行查询的请求
  - 例：如果**Student**表按专业**Smajor**的升序存放，则可以快速回答如下**SQL**语句

```
SELECT Sno, Sname, Ssex
FROM Student
WHERE Smajor='计算机科学与技术'
```

- 顺序存储
  - 劣势：维护代价较高（在增改时需要移动已有的记录以保序）

- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - B+树存储
  - 哈希存储

- 多表聚簇存储
  - 不同表的元组聚簇存放在同一组块中

- 多表聚簇存储
  - 例：两个具有主外码的参照关系表**Student**和**SC**，按照学号**Sno**相等聚簇存放

块头					
记录0	20180001	李勇	男	2000-3-8	信息安全
记录1	20180001	81001	85	20192	81001-01
记录2	20180001	81002	96	20201	81002-01
记录3	20180001	81003	87	20202	81003-01
记录4	20180002	刘晨	女	1999-9-1	计算机科学与技术
记录5	20180002	81001	80	20192	81001-02
记录6	20180002	81002	98	20201	81002-01
……	20180002	81003	71	20202	81003-02
	20180003	王敏	女	2001-8-1	计算机科学与技术
	20180003	81001	81	20192	81001-01
	20180003	81002	76	20201	81002-02
	20180004	张立	男	2000-1-8	计算机科学与技术
	20180004	81001	56	20192	81001-02
	20180004	81002	97	20201	81002-02
	20180005	陈新奇	男	2001-11-1	信息管理与信息系统
	20180205	81003	68	20202	81003-01
	20180006	赵明	男	2000-6-12	数据科学与大数据技术
	20180007	王佳佳	女	2001-12-7	数据科学与大数据技术
	空闲空间				

物理块

❖ 多表聚簇存储的优劣

- 优势：
  - 减少连接操作带来的开销
    - 可快速回答如下查询：

```
SELECT SC.Sno, Sname, count(*)
FROM Student, SC
WHERE Student.Sno=SC.sno AND SC.Sno='20180001';
```

❖ 多表聚簇存储的优劣

- 劣势：
  - 降低某些查询的查询效率（同一表中的元组会分散在更多块中）
    - 如：

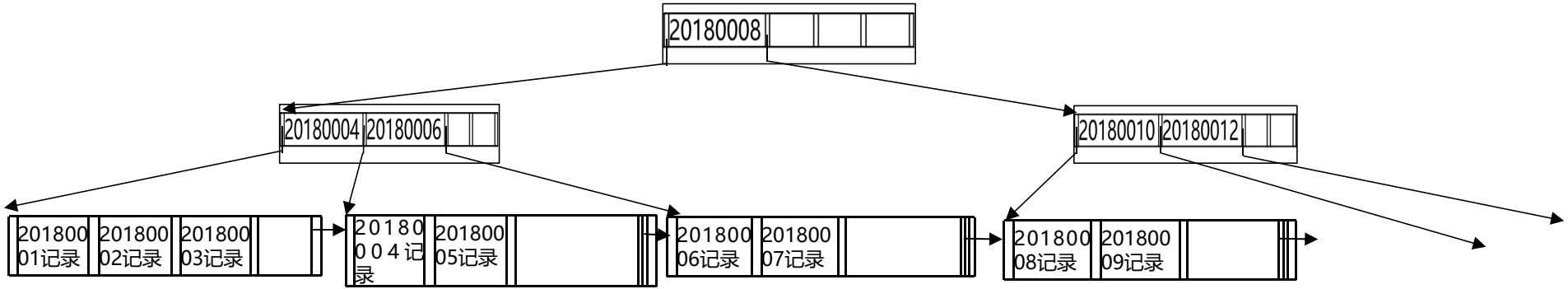
```
SELECT * FROM Student
WHERE Smajor=’数据科学与大数据技术’ AND Ssex=’女’;
```

  - 在更新操作时会带来更频繁的数据迁移
    - 如在新学期插入新的学生选课信息

- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - **B+树存储**
  - 哈希存储

❖ B+树存放方式

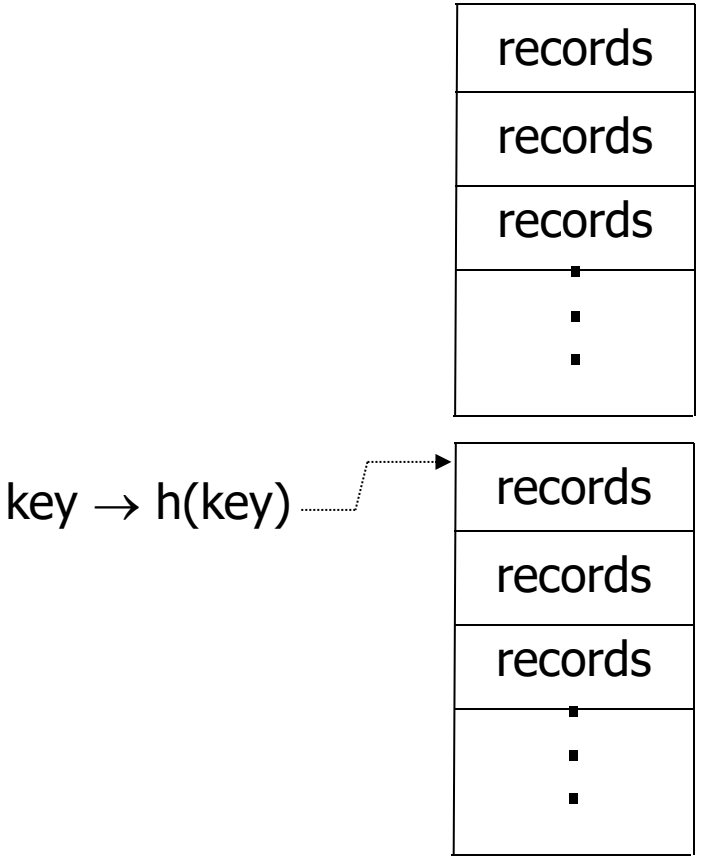
- 以B+树索引的方式确定记录存放在哪个数据块中
- 保持较高的访问效率的同时，降低数据维护开销



- 关系表的5种存放方式
  - 堆存储
  - 顺序存储
  - 多表聚簇存储
  - B+树存储
  - 哈希存储

❖哈希存放方式

- 用哈希函数计算表中指定属性的哈希值，以此确定相应记录放在哈希值中块中
  - 哈希表：由**B**个哈希桶组成，每个桶编号**0**到**B-1**，对应一个或多个物理块，存放一条或多条记录
  - 哈希函数：输入为记录的哈希属性，输出为介于**0**到**B-1**之间的整数，对应哈希桶号



- 哈希存储的优势
  - 将记录的存储位置与其指定属性的取值之间建立一个对应关系，查找记录时可快速定位这条记录

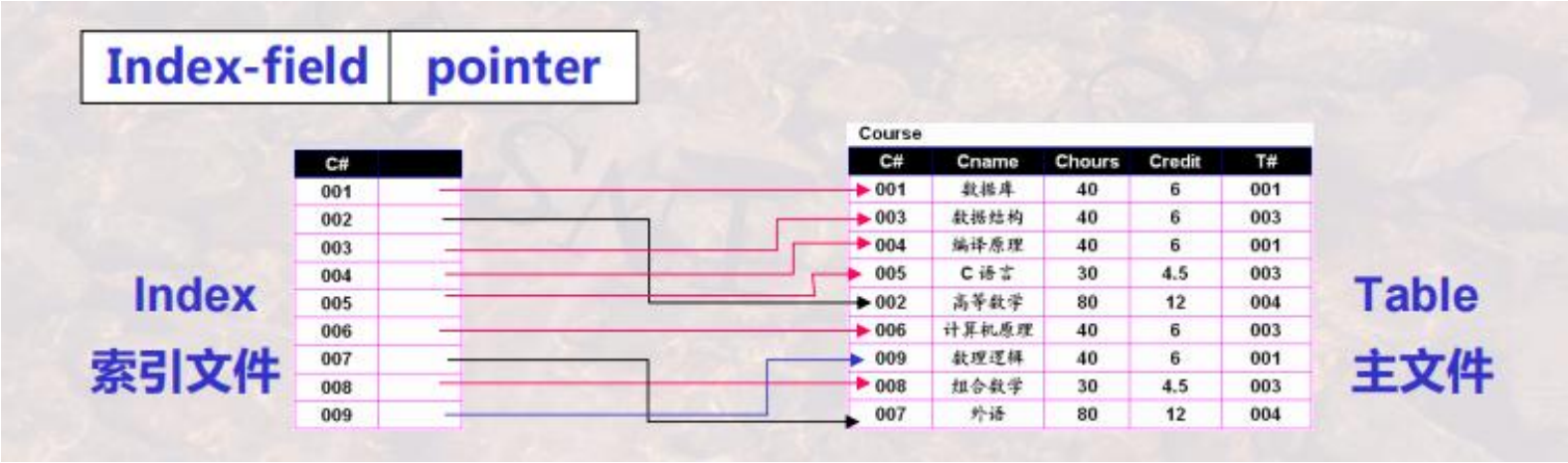
- ❖9.1 数据组织
- ❖9.2 索引结构
- ❖本章小结

- 用于从数据库中高效获取满足条件的记录
- SELECT * FROM library WHERE title = 'Database System' ;

文献卡片记录了书的基本信息与所在书架位置

文献卡片在卡片柜中以书名笔画数排序

- 建立索引的目的：加快查询速度
- 谁可以建立索引
  DBA 或 表的属主（即建立表的人）
  DBMS一般会自动建立索引
- 谁维护索引
  DBMS自动完成
- 使用索引
  DBMS自动选择是否使用索引以及使用哪些索引



❖ 几个关键点

1. 关系数据库为什么要有索引机制？
   - 层次数据库、网状数据库都没有索引机制
   - 为什么索引是关系数据库的重要实现技术

   → 提升效率

2. 索引为什么能提高效率
   - 表的索引块数量通常比数据块数量少得多
   - 可以有高效的方法快速查找索引块
   - 若索引文件足够小，可长期驻留内存缓冲区，减少I/O操作

❖ 几个关键点

3. 索引会带来额外的开销吗？
   - 存储索引开销
   - 建立索引开销
   - 维护索引开销
4. 索引能提高什么类型操作的效率
   - 查询？
   - 增删改？

- ❖9.2.1 顺序表索引
- ❖9.2.2 辅助索引
- ❖9.2.3 B+树索引
- ❖9.2.4 哈希索引
- ❖9.2.5 位图索引

❖ 顺序表索引

- 在顺序表的排序属性（组）上建立索引
- 同一个表的不同块通过指针链接实现有序
- 也称作主索引（**Primary Index**）或聚簇索引（**Clustering Index**）

语句格式

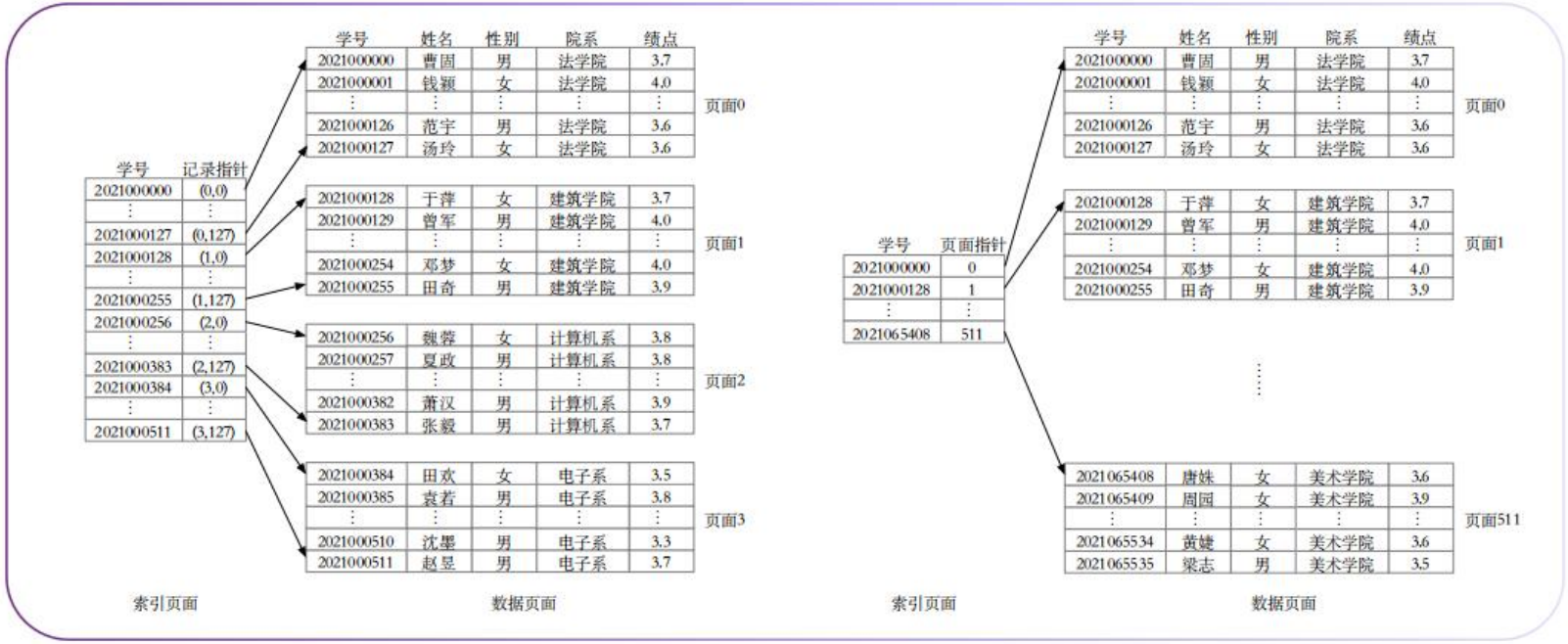
CREATE [UNIQUE] [CLUSTER] INDEX <索引名>

ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);

- 顺序表索引
  - 稠密索引
  - 稀疏索引
  - 多级索引

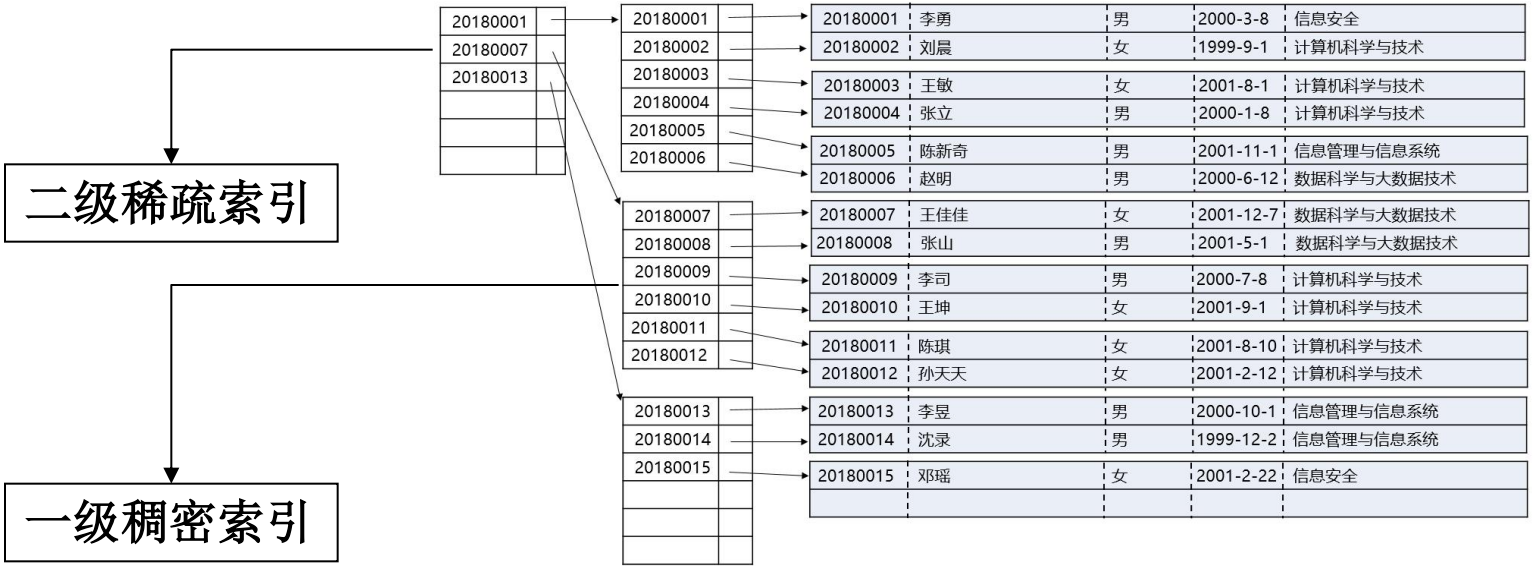
➢ 按照索引粒度分为：稠密索引 vs 稀疏索引

- 稠密索引：索引到数据记录
- 稀疏索引：索引到文件页面



❖多级索引（Multilevel Index）

■ 例：



- 多级索引（Multilevel Index）
  - 解决索引尺寸大问题
  - 索引建立
    - 第一级索引是稠密或稀疏索引
    - 第二级及以上为建立在上一级索引上的稀疏索引
    - 重复直到尺寸合适
  - 利用多级索引查找
    - 从高级索引逐层向下，知道定位到记录所在的物理块

- ❖9.2.1 顺序表索引
- ❖9.2.2 辅助索引
- ❖9.2.3 B+树索引
- ❖9.2.4 哈希索引
- ❖9.2.5 位图索引

- 辅助索引（**Secondary Index**）
  - 建立在表的非排序属性上的索引
  - 一个表最多只能建立一个主索引，但可以在不同属性上建立多个辅助索引
  - 辅助索引必须是稠密索引

❖辅助索引（Secondary Index）

- 建立在表的非排序属性上的索引

- 一个表最多只能建立一个主索引，但可以在不同属性上建立多个辅助索引
  - 只能按一个（组）属性排序
- 辅助索引必须是稠密索引

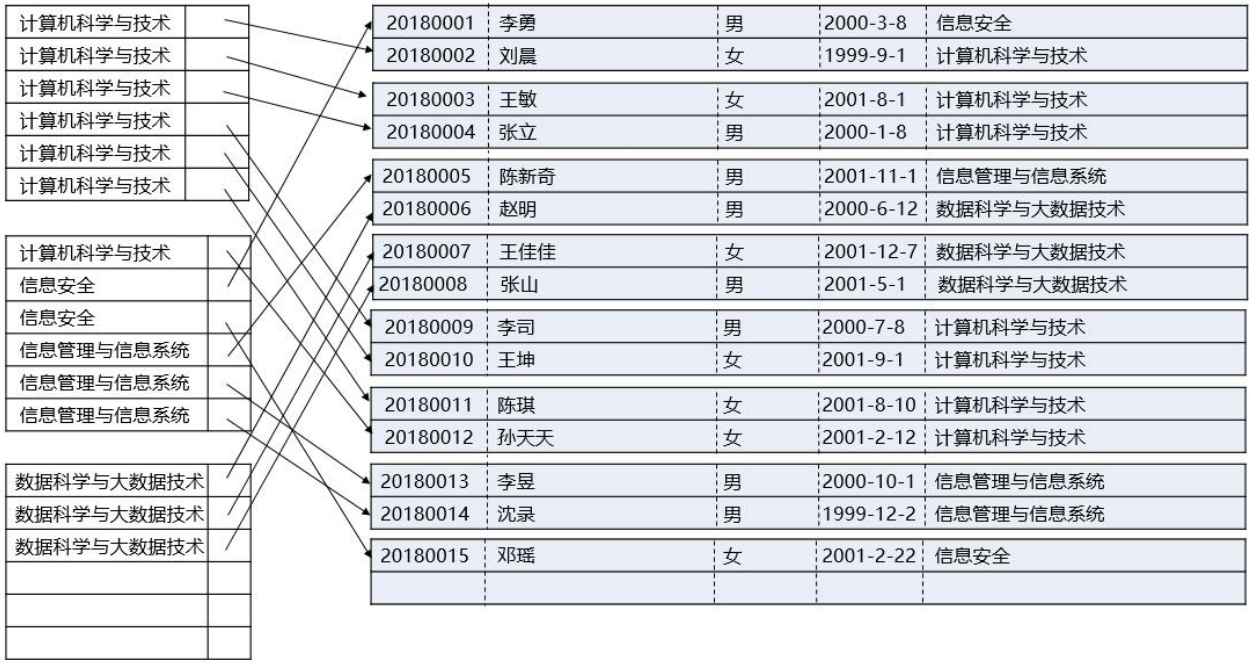
❖辅助索引（Secondary Index）

- 建立在表的非排序属性上的索引

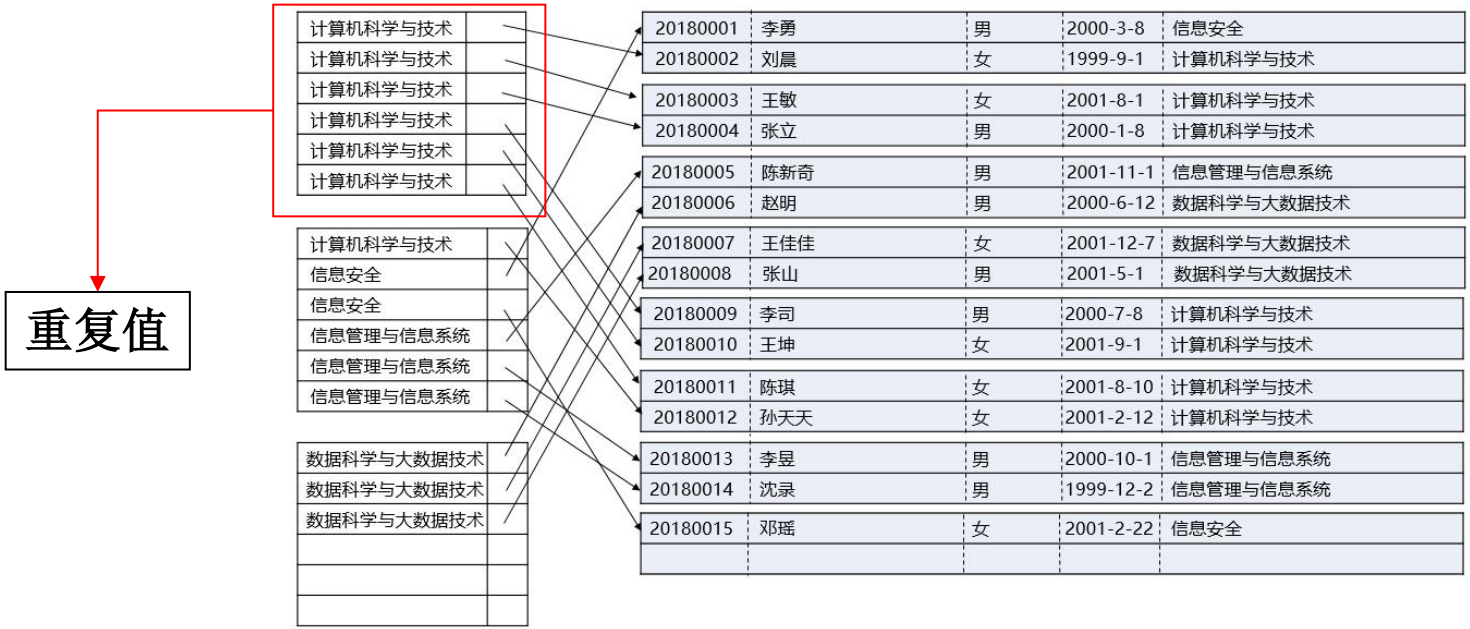
- 一个表最多只能建立一个主索引，但可以在不同属性上建立多个辅助索引
  - 只能按一个（组）属性排序
- 辅助索引必须是稠密索引
  - 辅助索引是建立在无序属性上的

❖辅助索引（Secondary Index）

■ 例：

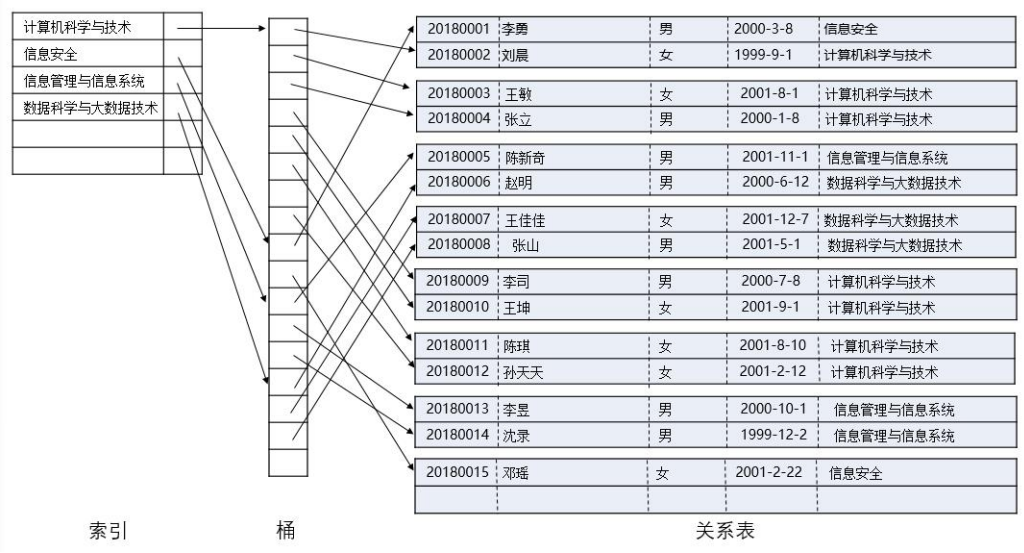


■ 辅助索引的属性往往会取重复值，去掉可减小索引大小与查找开销：



■ 引入指针桶去除重复索引项

- 索引项指针----指针桶相应位置----相应的元组
- 例：假设一个物理块放 6个索引项，16个指针桶，2条元组



❖辅助索引（Secondary Index）

- 当在关系表上建立了多个辅助索引时，可以利用指针桶回答涉及多个属性的查询
- 例：

```
SELECT * FROM Student
WHERE Smajor='计算机科学与技术'
AND Sbirthdate='2002-1-1';
```

  - 找出满足Smajor=‘计算机科学与技术’的指针组
  - 找出满足 Sbirthdate=‘2002-1-1’ 的指针组
  - 两个指针组求交集

- ❖9.2.1 顺序表索引
- ❖9.2.2 辅助索引
- ❖9.2.3 B+树索引
- ❖9.2.4 哈希索引
- ❖9.2.5 位图索引

- 稠密索引与稀疏索引
  - 随着数据量增大
    - 索引本身庞大，查找效率不能令人满意
    - 按同一属性的不同值查找，时间效率可能相差大
    - 为保持索引项和元组有序，维护代价高
- 多级索引
  - 缓解前两个问题
  - 加剧索引维护代价
- B+树索引
  - 解决大型索引的组织和维护
  - 查找效率高、按不同值查找性能平衡、易于维护等

- B+树索引
  - B+树索引的结构
  - B+树索引的查询
  - B+树索引的维护

- B+树索引
  - B+树索引的结构
  - B+树索引的查询
  - B+树索引的维护

- **B+树索引的结构**
  - 本质上是一个多级索引
  - 将索引块组织成一棵平衡树
    - 从树根到树叶的所有路径一样长

- **B+树的三类结点**
  - 根结点：只有一个
  - 中间结点
  - 叶结点

  <u>根结点与中间结点统称为非叶结点</u>

- **B+树的秩（order）**
  - 一个索引块最多能存放的指针的个数

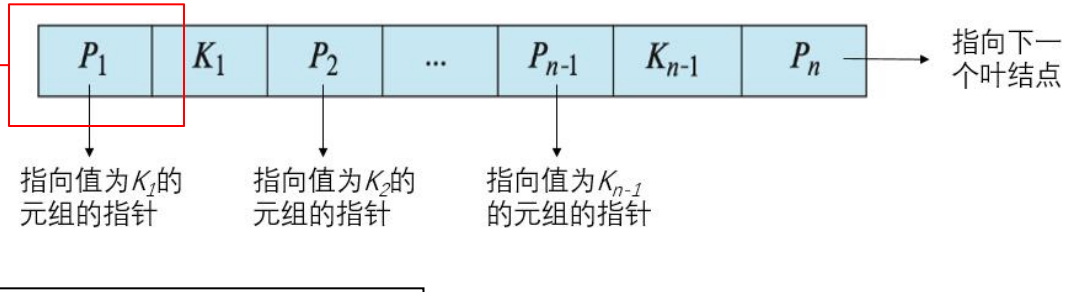
■ 例：B+树索引的典型结点

- 包含了n−1个属性值K_1，K_2，…，K_{n-1}
- n个指针P_1，P_2，…，P_n
- 结点中的属性值按序存放，如果$i<j$，则$K_i<K_j$

P_1	K_1	P_2	…	P_{n-1}	K_{n-1}	P_n

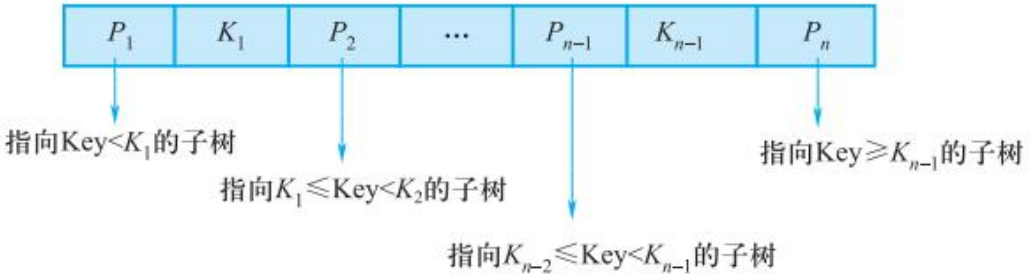
●B+树索引的叶结点

– 指针P_i指向关系表中属性值为K_i的元组（$i=1，2，…，n-1$）
– 指针P_n指向其兄弟叶结点。最后一个叶结点的P_n为空。

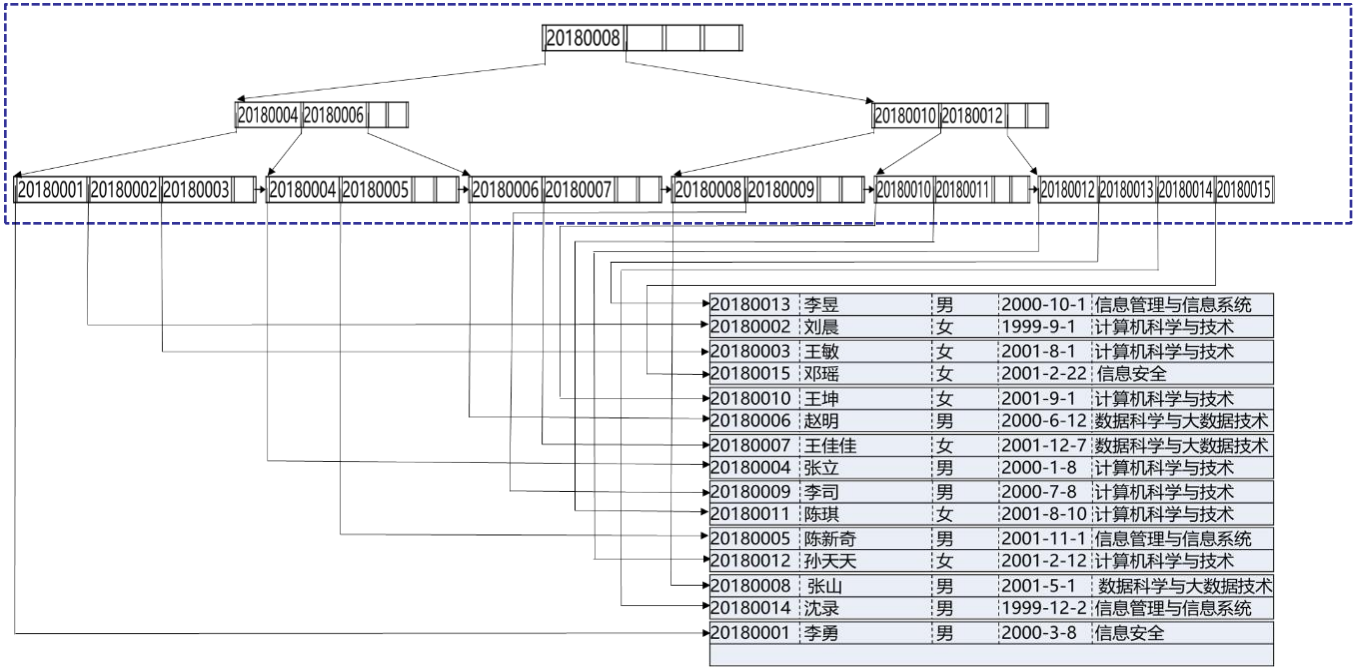


B+树存放方式中，B+树叶结点存放的是数据块
B+树索引中，B+树叶结点存放的是索引项

- **B+树索引的非叶结点**
  - ***P_i*指向其下层的孩子结点（*i=1，2，…，n*）。**
    - » ***P_1*指向的子树，其所有属性值*Key*均满足*Key* < *K_1***
    - » ***P_i*（*i=2，3，…，n-1*）指向的子树，其所有属性值*Key*均满足*K_{i-1}* ≤ *Key* < *K_i***
    - » ***P_n*指向的子树，其所有属性值*Key*均满足*Key* ≥ *K_{n-1}*。**



■ 例：Student表Sno属性上的B+树索引（n=5）

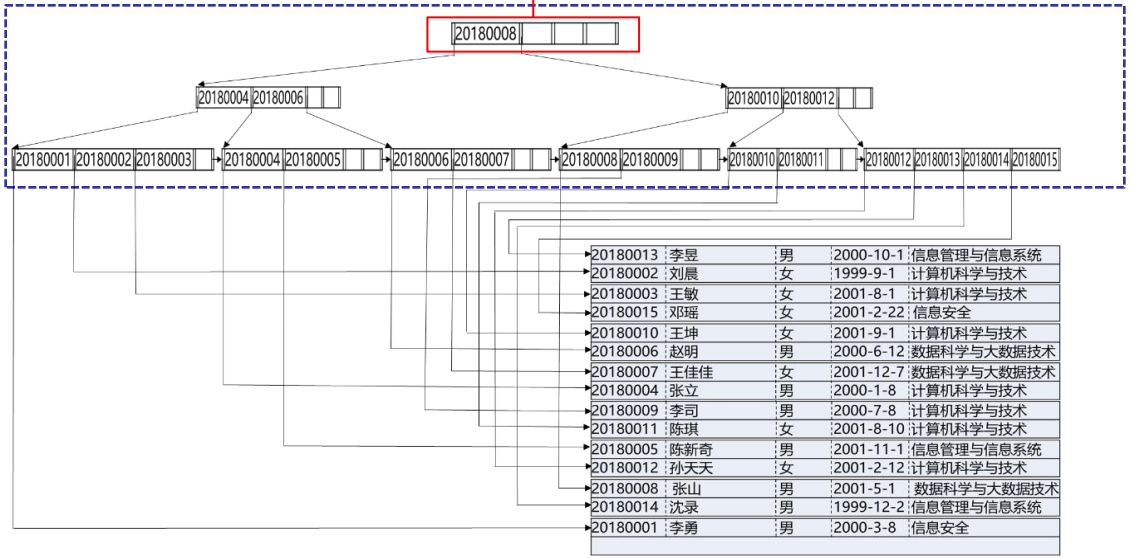


- B+树索引
  - B+树索引的结构
  - B+树索引的查询
  - B+树索引的维护

- **B+树索引的查询**
  - 高效地完成随机查找和范围查询
    - 随机查找：按照索引属性的取值进行查找
      - 从根结点开始
      - 沿父子结点指针逐层向下搜索
      - 直到叶结点（匹配则返回结果，否则没有满足条件的元组）

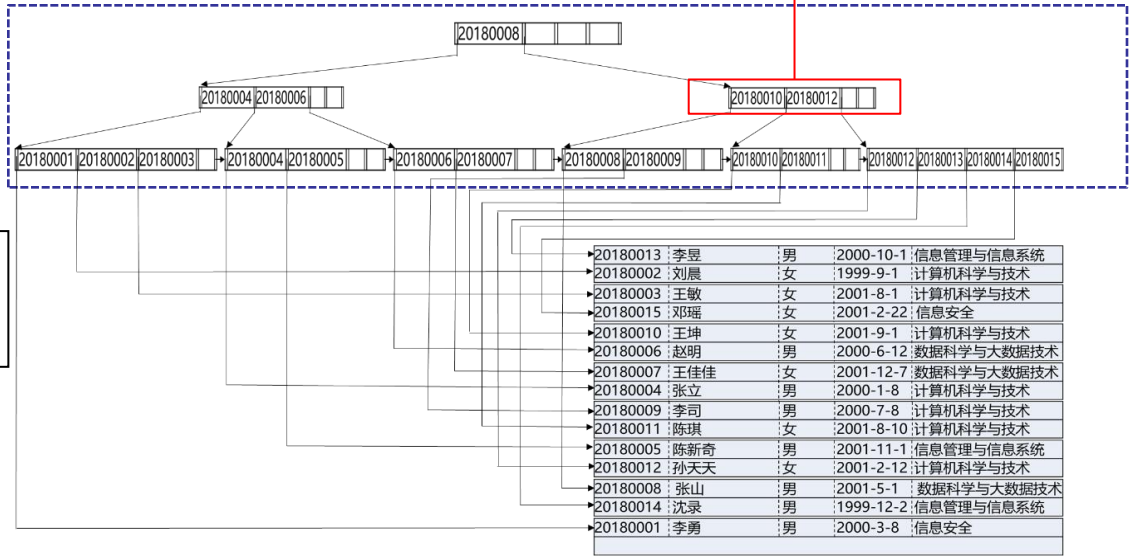
- 随机查找：按照索引属性的取值进行查找
  - 例：在Student表中查询*Sno=20180011*的学生信息

20180011>20180008
沿右子树继续搜索



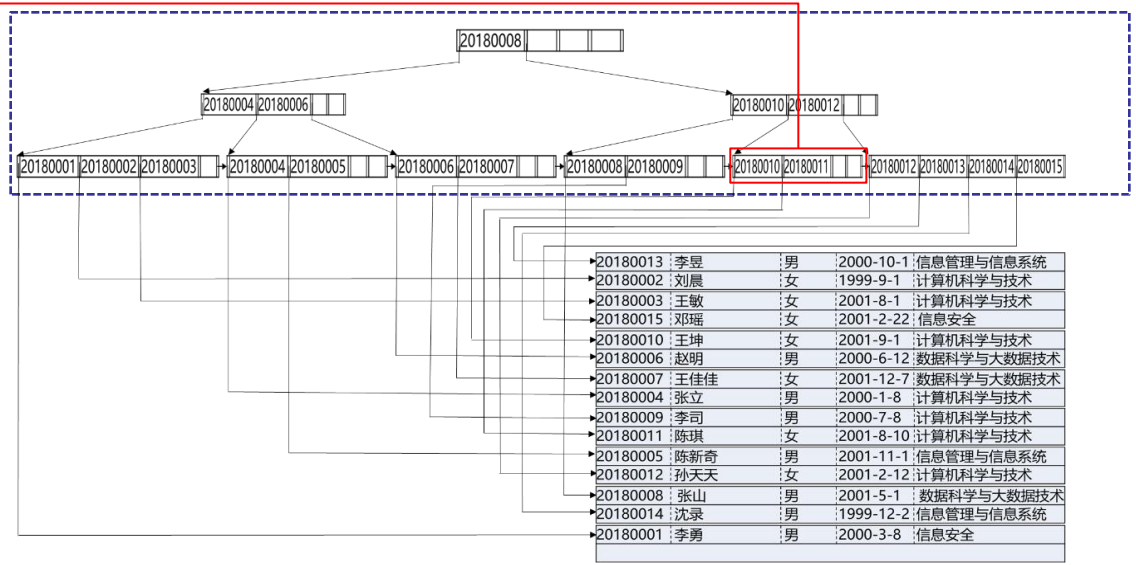
- 随机查找：按照索引属性的取值进行查找
  - 例：在Student表中查询*Sno=20180011*的学生信息

20180010<20180011<20180012
沿20180012左指针继续搜索



- 随机查找：按照索引属性的取值进行查找
  - 例：在Student表中查询*Sno=20180011*的学生信息

在叶结点中找到20180011
用其左侧的记录指针取出记录



- **B+树索引的查询**
  - 高效地完成随机查找和范围查询
    - 范围查找（**between … and …**）：
      - 用随机查找的方法分别找到范围条件的入口点和结束点
      - 对入口点和结束点之间的属性值进行顺序搜索

❖ B+树索引的查询

- 无论查询条件中的属性值是什么，查询效率都相似
  - 例：查询***Sno=20180011***的学生与查询***Sno=20180001***的学生，其**I/O**次数是一样的 （都是**4**次）

- B+树索引
  - B+树索引的结构
  - B+树索引的查询
  - B+树索引的维护

- 维护B+树索引的关键是如何维持B+树的平衡特性
  - 插入元组
  - 删除元组

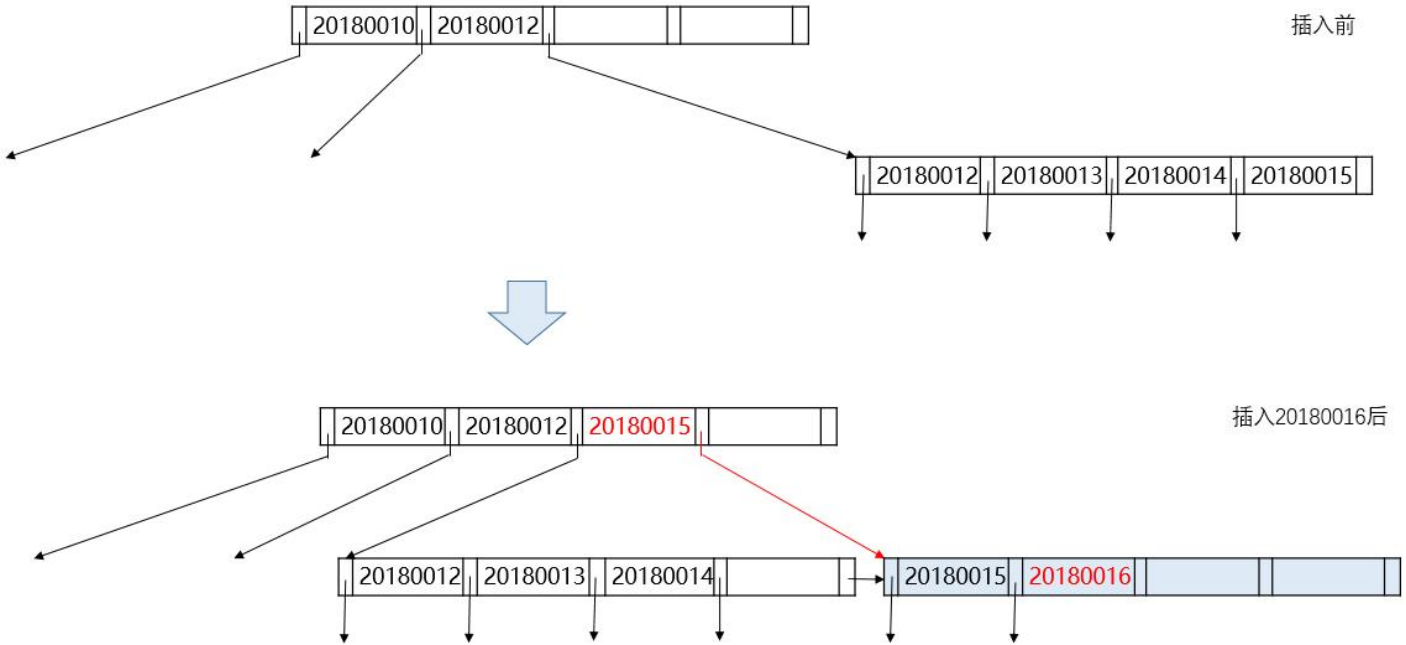
- 插入元组
  - 叶结点有空闲空间（**Key**值个数小于**n-1**）
    - 直接插入

❖插入元组

- 叶结点达到最大充满度（**key**值个数等于**n-1**）
  - 当前叶结点分裂成**2**个叶结点
  - 父结点插入
  - 维护父结点
  - 逐层向上，直到插入完成

<u>若根结点分裂会导致树的高度增加一层</u>

例：


20180010
20180012
插入前
20180012
20180013
20180014
20180015
20180010
20180012
20180015
插入20180016后
20180012
20180013
20180014
20180015
20180016

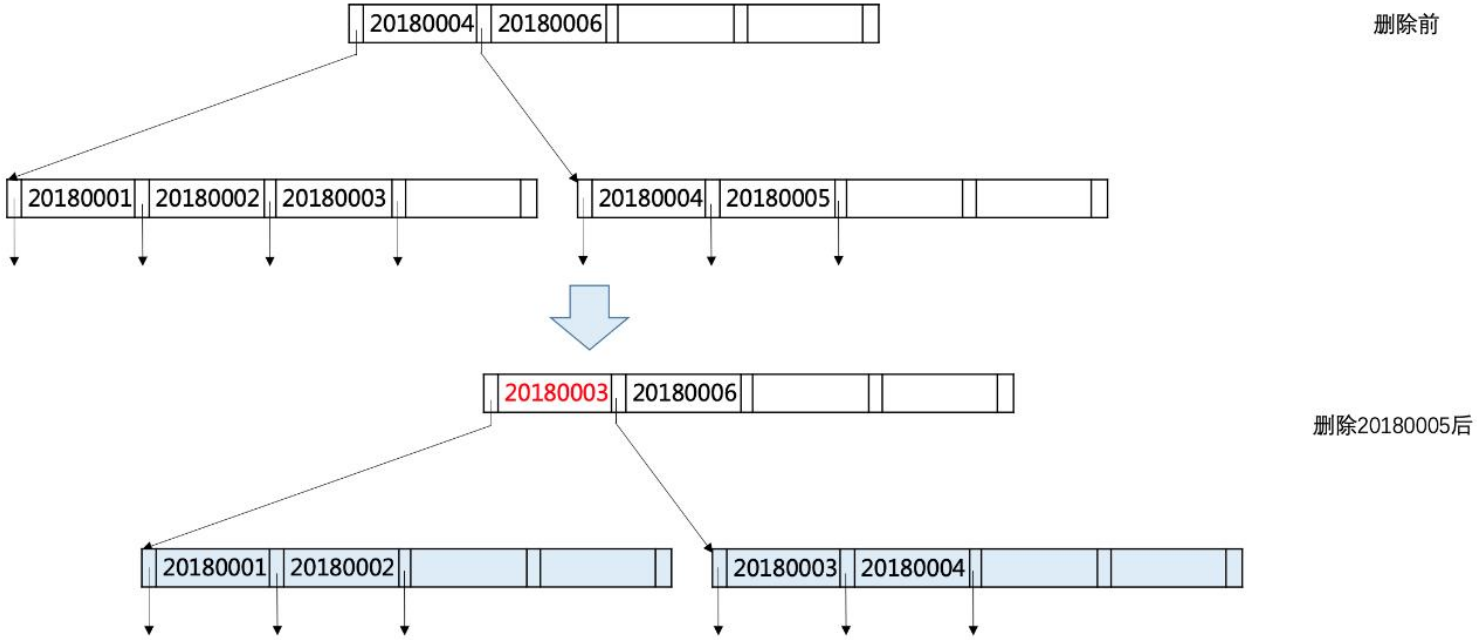
- 删除元组
  - 用随机搜索算法找到要删除的索引项
  - 删除后叶结点仍能满足最小充满度（**Key**值个数大于 $\lceil (n-1)/2 \rceil$ ）
    - 直接删除

■删除后叶结点不能满足最小充满度（Key值个数小于等于⌈(n−1)/2⌉）

- 删除当前索引项
- 合并叶结点
- 删除父结点索引项
- 维护父结点
- 逐层向上，直到删除完成

<u>若合并传递至根结点会导致树的高度降低一层</u>

■ 例：


20180004
20180006
删除前
20180001
20180002
20180003
20180004
20180005
20180003
20180006
删除20180005后
20180001
20180002
20180003
20180004

- ❖9.2.1 顺序表索引
- ❖9.2.2 辅助索引
- ❖9.2.3 B+树索引
- ❖9.2.4 哈希索引
- ❖9.2.5 位图索引

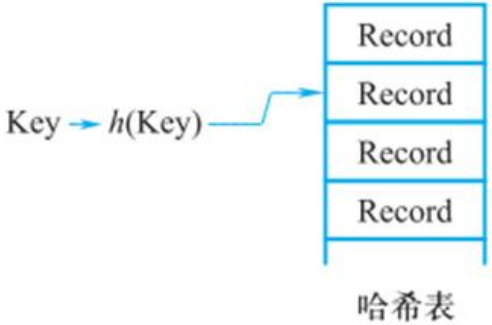
❖哈希索引

- 两个关键要素
  - 哈希表
    - B个哈希桶
  - 哈希函数
    - 将记录的索引属性值影射到哈希桶
    - 每个桶存放一条或多条哈希值相同的索引项
    - 每个索引项包括属性值和指向相应记录的指针

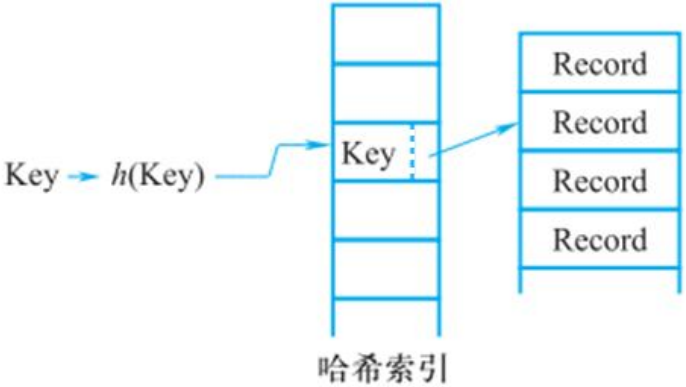
哈希文件组织中，哈希桶中存放的树数据记录

❖哈希索引

■ 例：



(a) 哈希存储方式



(b) 哈希索引

- **Hash索引（哈希索引）**
  - 静态哈希索引
  - 动态哈希索引

- **Hash索引（哈希索引）**
  - 静态哈希索引
  - 动态哈希索引

- 静态哈希索引
  - 哈希函数
  - 哈希表
  - 哈希索引的查找及维护

- 静态哈希索引
  - 哈希函数
  - 哈希表
  - 哈希索引的查找及维护

❖哈希函数的设计原则

- 尽量保证函数的取值随机和均匀
  - 不会出现某个桶的索引项远远超过其他桶

- 例：在Student表的Sno属性上建哈希索引（5000条元组，100个桶）
  - 学号转换为数值，除以100取模
  - 取学号后两位作为桶号

- 静态哈希索引
  - 哈希函数
  - 哈希表
  - 哈希索引的查找及维护

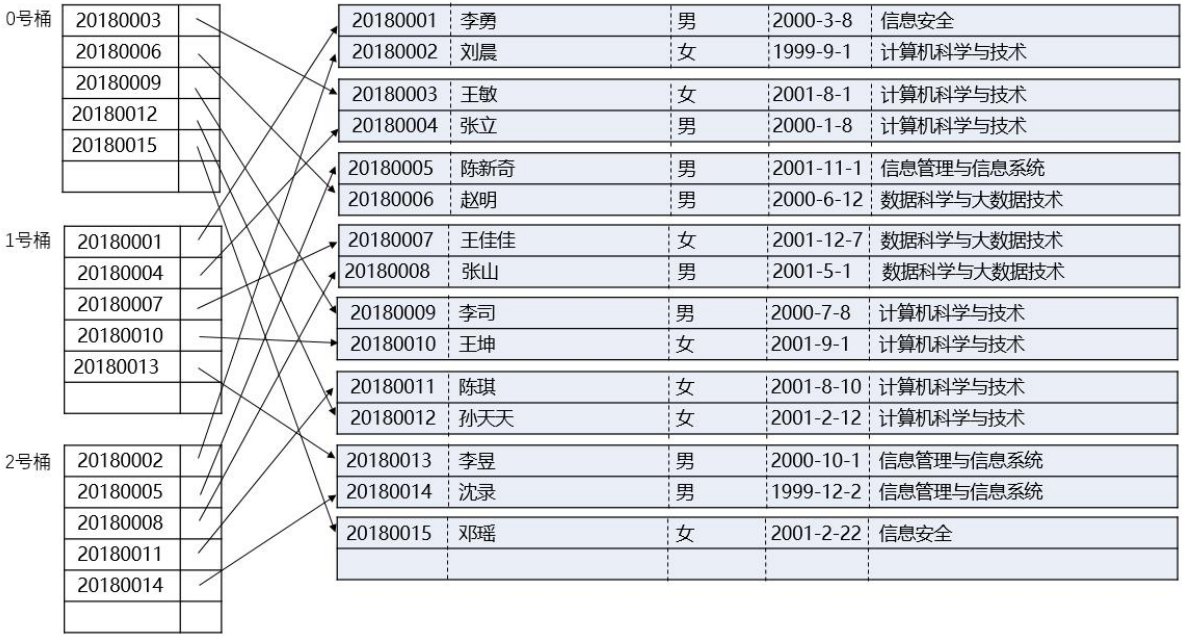
- 哈希表的构成
  - 由一组桶组成，一个桶对应一个或多个物理块
  - 桶中存放被映射到该桶的哈希索引项
  - 每条索引项包括索引属性值和指向相应记录的指针

❖哈希表的构成

- 例：Student表Sno属性上的哈希索引。
  - 每个桶最多可以存放6个索引项，共15条数据，设有3个哈希桶。
  - 哈希函数是取学号最后2位，将其转换数值型，除以3并取模作为桶号

❖哈希表的构成

■ 例：

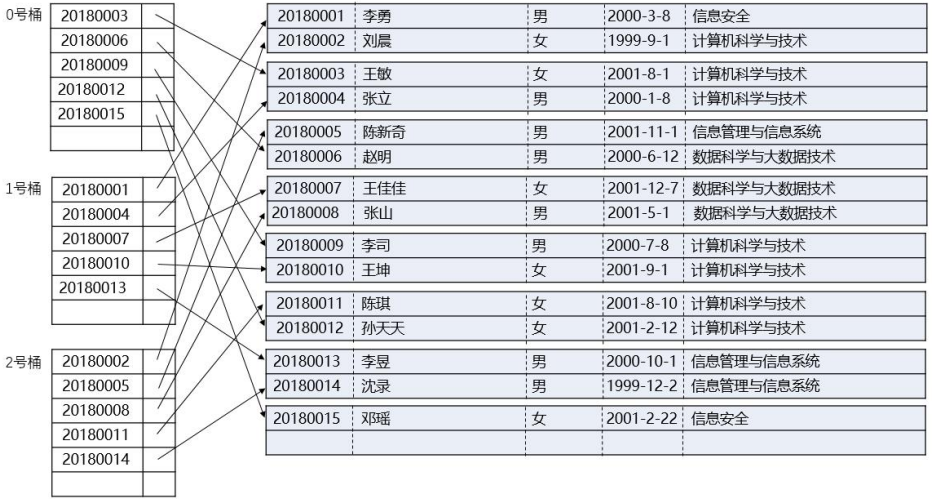

0号桶
20180003
20180006
20180009
20180012
20180015
1号桶
20180001
20180004
20180007
20180010
20180013
2号桶
20180002
20180005
20180008
20180011
20180014
20180001 李勇 男 2000-3-8 信息安全
20180002 刘晨 女 1999-9-1 计算机科学与技术
20180003 王敏 女 2001-8-1 计算机科学与技术
20180004 张立 男 2000-1-8 计算机科学与技术
20180005 陈新奇 男 2001-11-1 信息管理与信息系统
20180006 赵明 男 2000-6-12 数据科学与大数据技术
20180007 王佳佳 女 2001-12-7 数据科学与大数据技术
20180008 张山 男 2001-5-1 数据科学与大数据技术
20180009 李司 男 2000-7-8 计算机科学与技术
20180010 王坤 女 2001-9-1 计算机科学与技术
20180011 陈琪 女 2001-8-10 计算机科学与技术
20180012 孙天天 女 2001-2-12 计算机科学与技术
20180013 李昱 男 2000-10-1 信息管理与信息系统
20180014 沈录 男 1999-12-2 信息管理与信息系统
20180015 邓瑶 女 2001-2-22 信息安全

- 桶溢出：
  - 某个桶的存储空间不足

❖桶溢出的三类主要原因：

■哈希桶数量不足，不能存放所有的索引项

●例：下图中元组数量超过18条

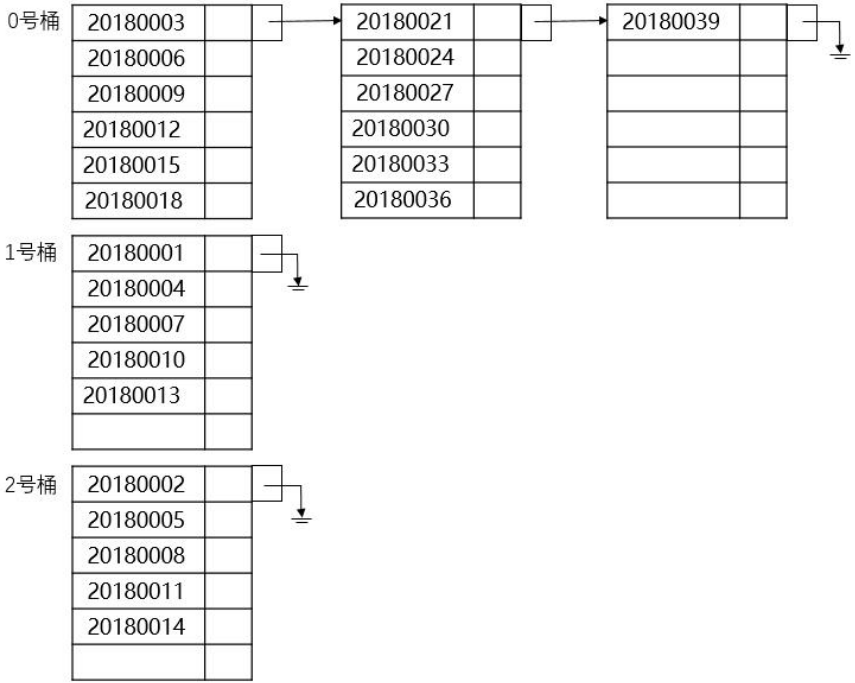


❖ 桶溢出的三类主要原因：

- 属性存在偏斜，某些属性值过多
  - 例：**Student**表中，‘计算机科学与技术’专业的学生人数非常多，使得相应的桶无法容纳下所有的索引项
- 哈希函数设计不合理，无法将索引均匀地映射到每个桶，导致某个桶的数据过多

- 减少桶溢出的措施：
  - 预留一定百分比的空间
    - 溢出仍不可避免
  - 分配溢出桶
    - 当溢出桶空间充满时，追加新的溢出桶
    - 用溢出链将一个桶及其溢出桶链接在一起

■ 溢出桶示例：


0号桶
20180003
20180006
20180009
20180012
20180015
20180018
20180021
20180024
20180027
20180030
20180033
20180036
20180039
1号桶
20180001
20180004
20180007
20180010
20180013
2号桶
20180002
20180005
20180008
20180011
20180014

- 静态哈希索引
  - 哈希函数
  - 哈希表
  - 哈希索引的查找及维护

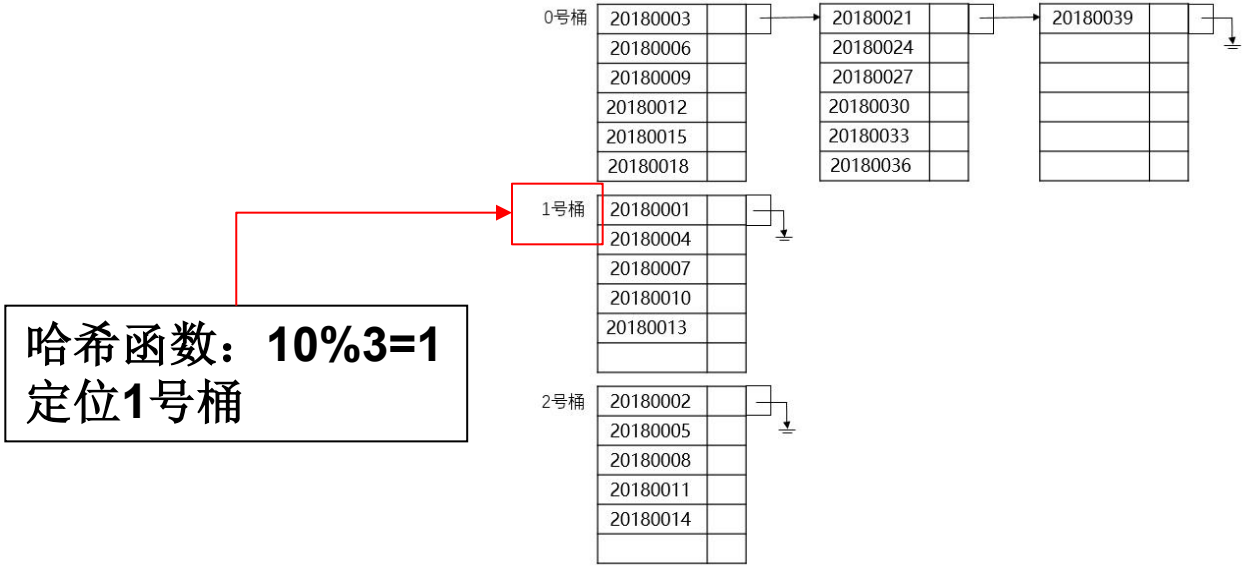
- 哈希索引的查找及维护
  - 哈希索引的查找
  - 哈希索引的维护

- 哈希索引的查找及维护
  - 哈希索引的查找
  - 哈希索引的维护

- 哈希索引的查找
  - 擅长做等值查找
    - 根据属性值计算出哈希函数值，得到桶号
    - 到相应的桶中搜索相应的索引项
    - 若桶中没有找到，但存在溢出桶，继续搜索溢出桶

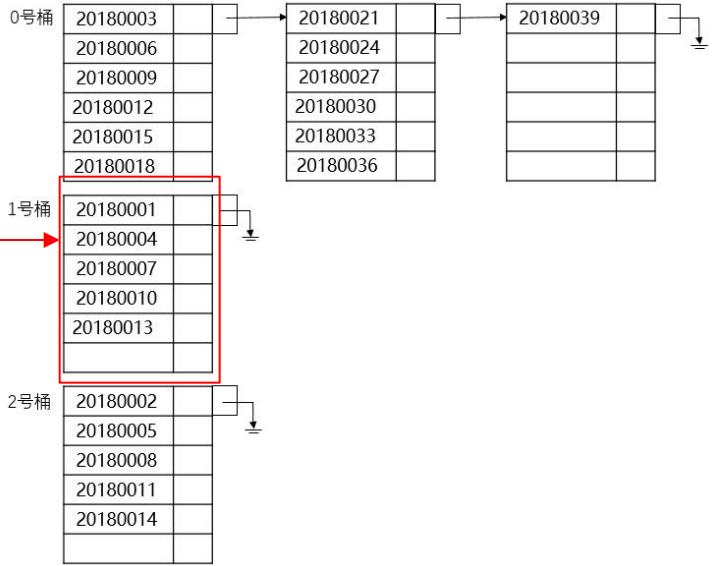
❖哈希索引的查找

■ 例：查找学号为20180010的学生



❖哈希索引的查找

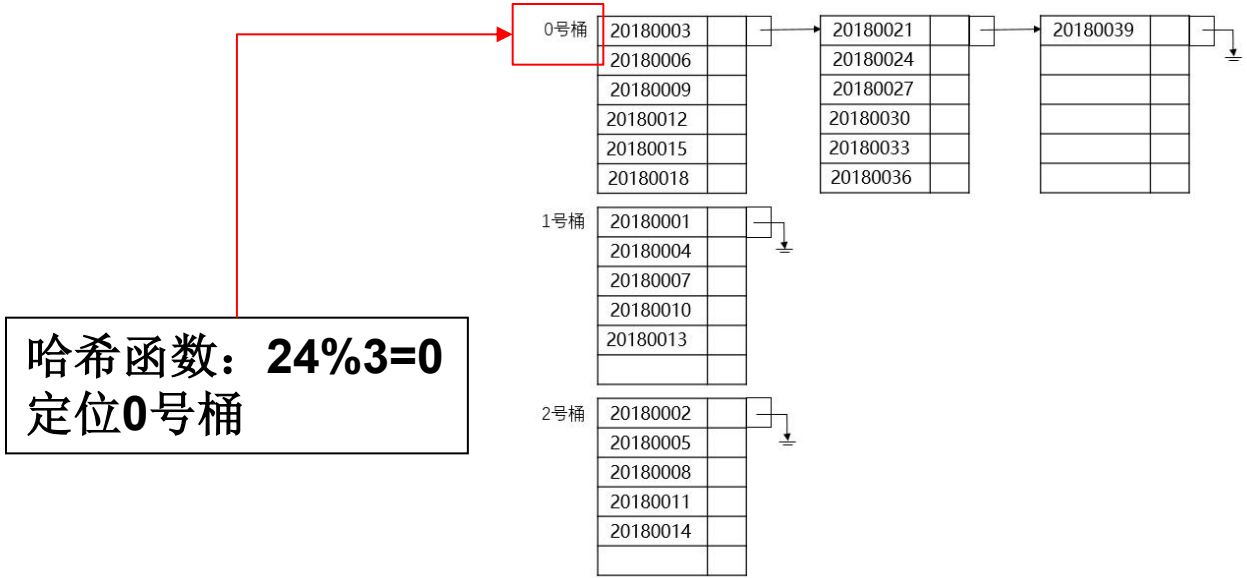
■ 例：查找学号为20180010的学生



读取1号桶
查找20180010索引项
根据索引项读取数据

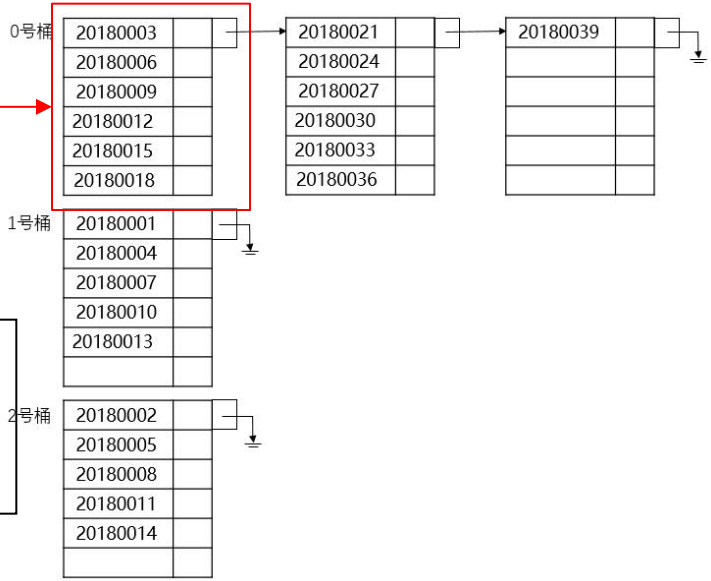
❖哈希索引的查找

■ 例：查找学号为20180024的学生



❖哈希索引的查找

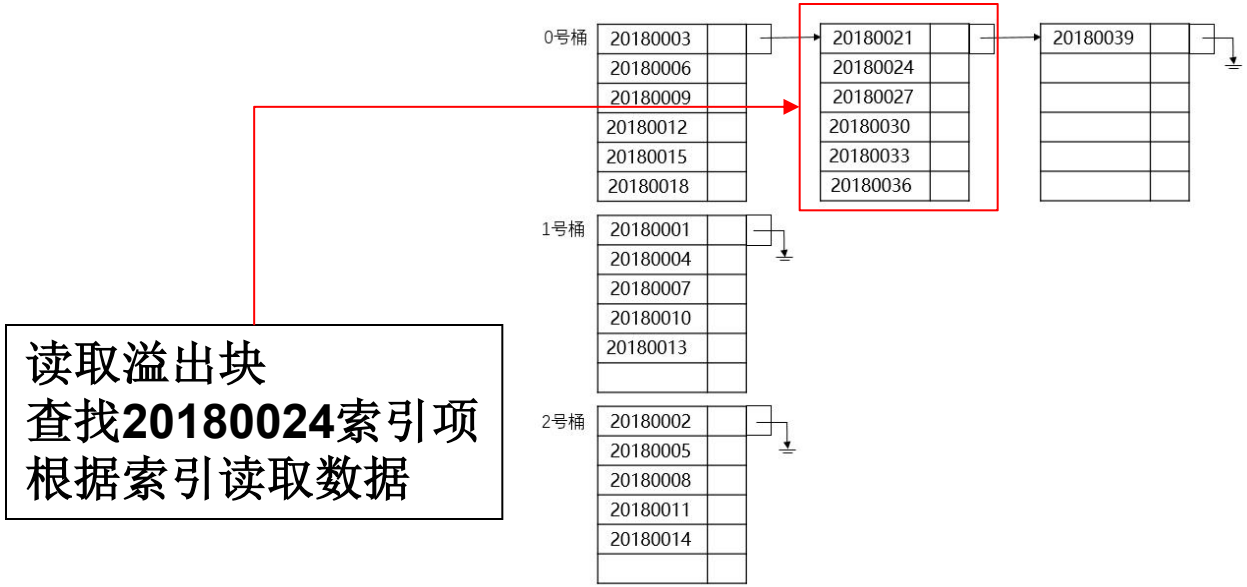
■ 例：查找学号为20180024的学生



读取0号桶
未找到20180024索引项
定位溢出块

❖哈希索引的查找

■ 例：查找学号为20180024的学生



- 哈希索引的查找及维护
  - 哈希索引的查找
  - 哈希索引的维护

- 哈希索引的维护
  - 插入元组时，需向哈希索引中插入相应的索引项
    - 哈希函数计算桶号
      - 若桶中有空间，直接插入
      - 若桶中空间已满，申请溢出桶并插入

- 哈希索引的维护
  - 删除元组时，需删除相应的索引项
    - 哈希函数计算桶号
      - 若没有溢出块，直接删除
      - 若有溢出块，判断是否有空间合并溢出块

- 哈希索引（哈希索引）
  - 静态哈希索引
  - 动态哈希索引

- 静态哈希索引中，桶的个数事先确定且不再改变
  - 桶溢出不可避免，影响查找效率

- 两种解决方法
  - 适当预留桶空间
    - 表数据量难以估算
  - 随着关系表的增大，周期性地增加哈希桶数目，并重组索引
    - 耗时，影响正在进行的查询

- 动态哈希索引
  - 随着关系表的增大，逐渐扩大桶的数目
  - 两类动态哈希索引
    - 可扩展哈希索引
    - 线性哈希索引

- ❖9.2.1 顺序表索引
- ❖9.2.2 辅助索引
- ❖9.2.3 B+树索引
- ❖9.2.4 哈希索引
- ❖9.2.5 位图索引

- 位图索引
  - 长度为***n***的位向量集合
    - ***n***：索引属性的基数（可能取值的个数）
    - 每一个位向量对应于索引属性的一个可能的取值

- 位图索引
  - 如果第*i*条记录的索引属性值为*v*，那么对应于值*v*的位向量在位置*i*上取值为1，其他的位向量在位置*i*上取值为0

❖位图索引

- 例：在**Student**表的**Ssex**属性上建立位图索引
  - 性别属性**Ssex**只有“男”和“女”两个可能的取值，基数为**2**，因而**Ssex**属性上的位图索引为长度为**2**的位向量集合

❖位图索引

■ 例：

Ssno	Sname	Ssex	Sbirthdate	Smajor	男	女
20180001	李勇	男	2000-3-8	信息安全	1	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1
20180004	张立	男	2000-1-8	计算机科学与技术	1	0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	1	0
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	1	0
20180009	李司	男	2000-7-8	计算机科学与技术	1	0
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1
20180013	李昱	男	2000-10-1	信息管理与信息系统	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001-2-22	信息安全	0	1

10对应男生

01对应女生

❖位图索引

- 例：查询**Student**表的所有女生

```
SELECT * FROM Student
WHERE Ssex='女';
```

- 借助**Ssex**属性上的位图索引，取出第**2**位为**1**的所有元组

❖位图索引

- 例：统计**Student**表上男生和女生的人数

```
SELECT Ssex, Count(*)
FROM Student
GROUP By Ssex;
```

  - 分别统计**Ssex**属性上位图索引各个位中**1**的个数（无需访问基本表）

❖位图索引

■ 例：在**Student**表的**Smajor**上也建有一个索引

Ssno	Sname	Ssex	Sbirthdate	Smajor	信息安全	计算机科学与技术	信息管理与信息系统	数据科学与大数据技术
20180001	李勇	男	2000-3-8	信息安全	1	0	0	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1	0	0
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1	0	0
20180004	张立	男	2000-1-8	计算机科学与技术	0	1	0	0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	0	0	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	0	0	0	1
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0	0	0	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	0	0	0	1
20180009	李司	男	2000-7-8	计算机科学与技术	0	1	0	0
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1	0	0
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1	0	0
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1	0	0
20180013	李昱	男	2000-10-1	信息管理与信息系统	0	0	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	0	0	1	0
20180015	邓瑶	女	2001-2-22	信息安全	1	0	0	0

❖位图索引

■ 例：统计计算机科学与技术专业男生人数。

```
SELECT count(*)
FROM Student
WHERE Ssex='男' AND Smajor='计算机科学与技术';
```

- 将**Ssex**属性位图索引的第**1**位与**Smajor**属性位图索引的第**2**位进行与操作，得到结果向量：

  (0, 0, 0,1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0)

- 对结果向量中**1**的个数进行计数，结果为**2**

与运算： (0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0)

Ssno	Sname	Ssex	Sbirthdate	Smajor	男	女
20180001	李勇	男	2000–3–8	信息安全	1	0
20180002	刘晨	女	1999–9–1	计算机科学与技术	0	1
20180003	王敏	女	2001–8–1	计算机科学与技术	0	1
20180004	张立	男	2000–1–8	计算机科学与技术	1	0
20180005	陈新奇	男	2001–11–1	信息管理与信息系统	1	0
20180006	赵明	男	2000–6–12	数据科学与大数据技术	1	0
20180007	王佳佳	女	2001–12–7	数据科学与大数据技术	0	1
20180008	张山	男	2001–5–1	数据科学与大数据技术	1	0
20180009	李司	男	2000–7–8	计算机科学与技术	1	0
20180010	王坤	女	2001–9–1	计算机科学与技术	0	1
20180011	陈琪	女	2001–8–10	计算机科学与技术	0	1
20180012	孙天天	女	2001–2–12	计算机科学与技术	0	1
20180013	李昱	男	2000–10–1	信息管理与信息系统	1	0
20180014	沈录	男	1999–12–2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001–2–22	信息安全	0	1

Ssno	Sname	Ssex	Sbirthdate	Smajor	信息安全	计算机科学与技术	信息管理与信息系统	数据科学与大数据技术
20180001	李勇	男	2000–3–8	信息安全	1	0	0	0
20180002	刘晨	女	1999–9–1	计算机科学与技术	0	1	0	0
20180003	王敏	女	2001–8–1	计算机科学与技术	0	1	0	0
20180004	张立	男	2000–1–8	计算机科学与技术	0	1	0	0
20180005	陈新奇	男	2001–11–1	信息管理与信息系统	0	0	1	0
20180006	赵明	男	2000–6–12	数据科学与大数据技术	0	0	0	1
20180007	王佳佳	女	2001–12–7	数据科学与大数据技术	0	0	0	1
20180008	张山	男	2001–5–1	数据科学与大数据技术	0	0	0	1
20180009	李司	男	2000–7–8	计算机科学与技术	0	1	0	0
20180010	王坤	女	2001–9–1	计算机科学与技术	0	1	0	0
20180011	陈琪	女	2001–8–10	计算机科学与技术	0	1	0	0
20180012	孙天天	女	2001–2–12	计算机科学与技术	0	1	0	0
20180013	李昱	男	2000–10–1	信息管理与信息系统	0	0	1	0
20180014	沈录	男	1999–12–2	信息管理与信息系统	0	0	1	0
20180015	邓瑶	女	2001–2–22	信息安全	1	0	0	0

❖位图索引

- 有效处理多值查询
- 例：

```
SELECT *
FROM Student
WHERE Smajor in ('数据科学与大数据技术', '信息安全');
```

- 对**Smajor**属性的位图索引的第**1**位和第**4**位进行或操作，得到结果向量：

(1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1)

或运算： (1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1)

Ssno	Sname	Ssex	Sbirthdate	Smajor	信息安全	计算机科学与技术	信息管理与信息系统	数据科学与大数据技术
20180001	李勇	男	2000-3-8	信息安全	1	0	0	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1	0	0
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1	0	0
20180004	张立	男	2000-1-8	计算机科学与技术	0	1	0	0
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	0	0	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	0	0	0	1
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	0	0	0	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	0	0	0	1
20180009	李司	男	2000-7-8	计算机科学与技术	0	1	0	0
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1	0	0
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1	0	0
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1	0	0
20180013	李昱	男	2000-10-1	信息管理与信息系统	0	0	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	0	0	1	0
20180015	邓瑶	女	2001-2-22	信息安全	1	0	0	0

❖位图索引的优劣

- 优势：
  - 使用位操作来快速回答用户查询或快速定位满足条件的元组集合，减少对基本表的全表扫描，提高查询效率
- 劣势：
  - 位图索引的大小与列的基数成正比，基数大的列其位图索引会非常庞大，因此它只适用于基数小的属性列

- 编码位图索引（**Encoded Bitmap Index** ）
  - 改进标准位图索引
    - 通过对属性值编码，减少索引位向量的个数，从而能够应对有较高基数的列

❖编码位图索引（Encoded Bitmap Index ）

■ 例：Student表Smajor属性的编码位图索引

Ssno	Sname	Ssex	Sbirthdate	Smajor		
20180001	李勇	男	2000-3-8	信息安全	0	0
20180002	刘晨	女	1999-9-1	计算机科学与技术	0	1
20180003	王敏	女	2001-8-1	计算机科学与技术	0	1
20180004	张立	男	2000-1-8	计算机科学与技术	0	1
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	1	0
20180006	赵明	男	2000-6-12	数据科学与大数据技术	1	1
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	1	1
20180008	张山	男	2001-5-1	数据科学与大数据技术	1	1
20180009	李司	男	2000-7-8	计算机科学与技术	0	1
20180010	王坤	女	2001-9-1	计算机科学与技术	0	1
20180011	陈琪	女	2001-8-10	计算机科学与技术	0	1
20180012	孙天天	女	2001-2-12	计算机科学与技术	0	1
20180013	李昱	男	2000-10-1	信息管理与信息系统	1	0
20180014	沈录	男	1999-12-2	信息管理与信息系统	1	0
20180015	邓瑶	女	2001-2-22	信息安全	0	0

编码映射表

信息安全	00
计算机科学与技术	01
信息管理与信息系统	10
数据科学与大数据技术	11

- ❖9.1 数据组织
- ❖9.2 索引结构
- ❖本章小结

❖本章主要内容

- 关系数据库的数据组织方式
  - 逻辑组织方式
  - 物理组织方式：记录、块和关系表
- 索引结构：顺序表索引、辅助索引、B+树索引、哈希索引和位图索引